SEMANA 12 · CACHÉ Y DISTRIBUCIÓN DE CONTENIDO

Por qué cachear: latencia y jerarquía de caché

Parte 1 de 4 · Unidad 3

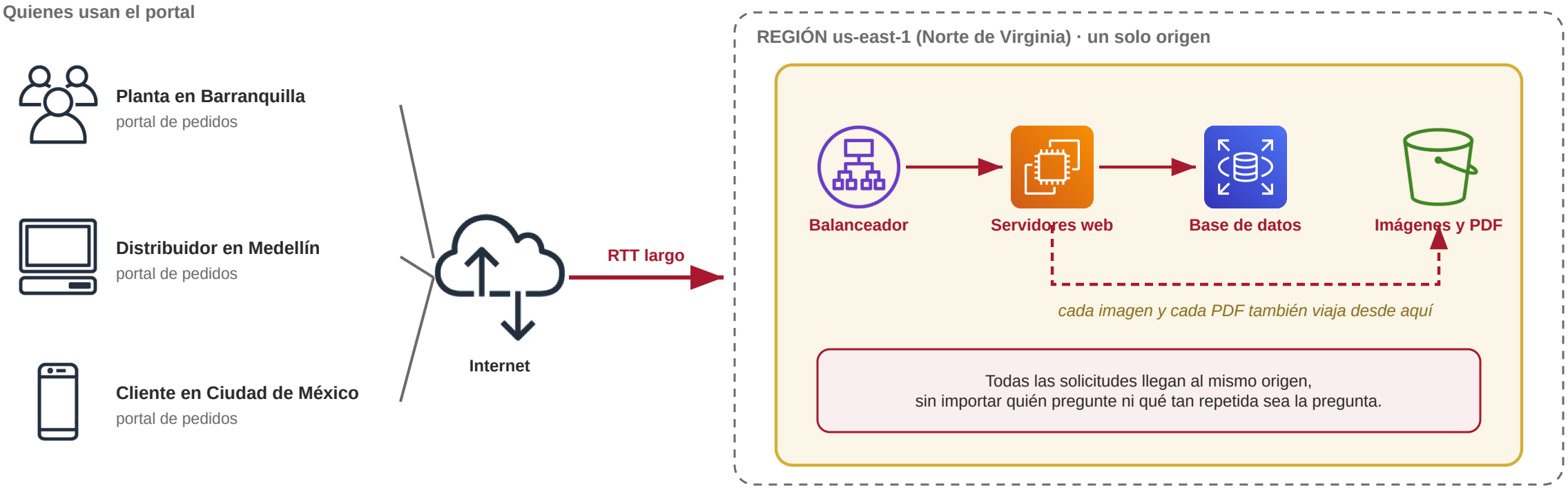
Ing. Rodolfo Cañas Cervantes
Universidad de la Costa (CUC) · Ingeniería de Sistemas
Semana 12 · Periodo 2026-2 · Barranquilla, Colombia

Entiende de dónde sale la lentitud de una aplicación y cómo una copia cercana la elimina.

ENCUADRE

El portal de la Corporación es lento fuera de Barranquilla

La Corporación Industrial del Caribe sirve todo desde un solo origen, y la lentitud la sienten sus clientes lejanos.



4,8 s
en cargar el catálogo desde Ciudad de México (ilustrativo)

38
solicitudes HTTP por página: HTML, CSS, JS e imágenes

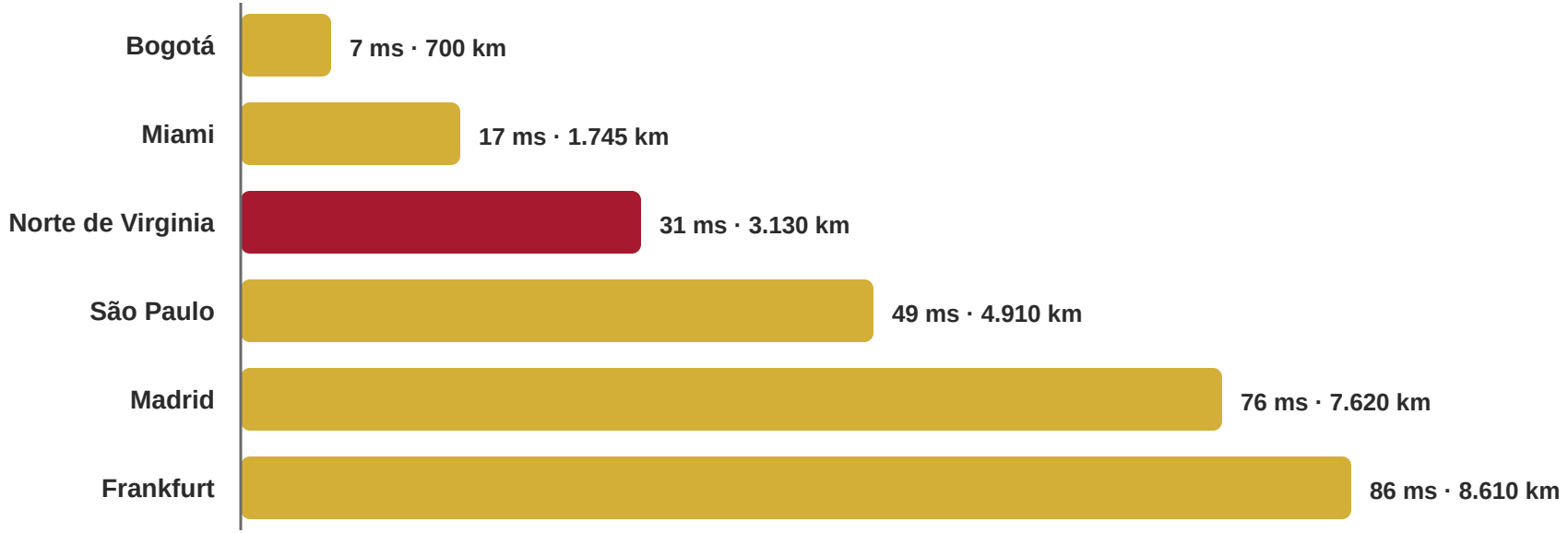
3,2 MB
que se descargan en cada visita, casi todo repetido

Fuente: caso docente Corporación Industrial del Caribe · cifras ilustrativas · íconos AWS

La distancia fija un piso a la latencia

Ninguna optimización del servidor baja de lo que cuesta la física: la salida es acercar el contenido.

Tiempo mínimo de ida y vuelta por fibra óptica desde Barranquilla (ms)



La cuenta que nadie puede evitar

Luz en fibra ≈ 200.000 km/s → 100 km de distancia cuestan cerca de 1 ms de ida y vuelta.

Una página real necesita varios viajes: resolver el nombre, abrir TCP, negociar TLS, pedir el HTML y luego cada recurso.

10 viajes × 31 ms ≈ 310 ms perdidos solo en distancia, antes de que el servidor haga una sola línea de trabajo.

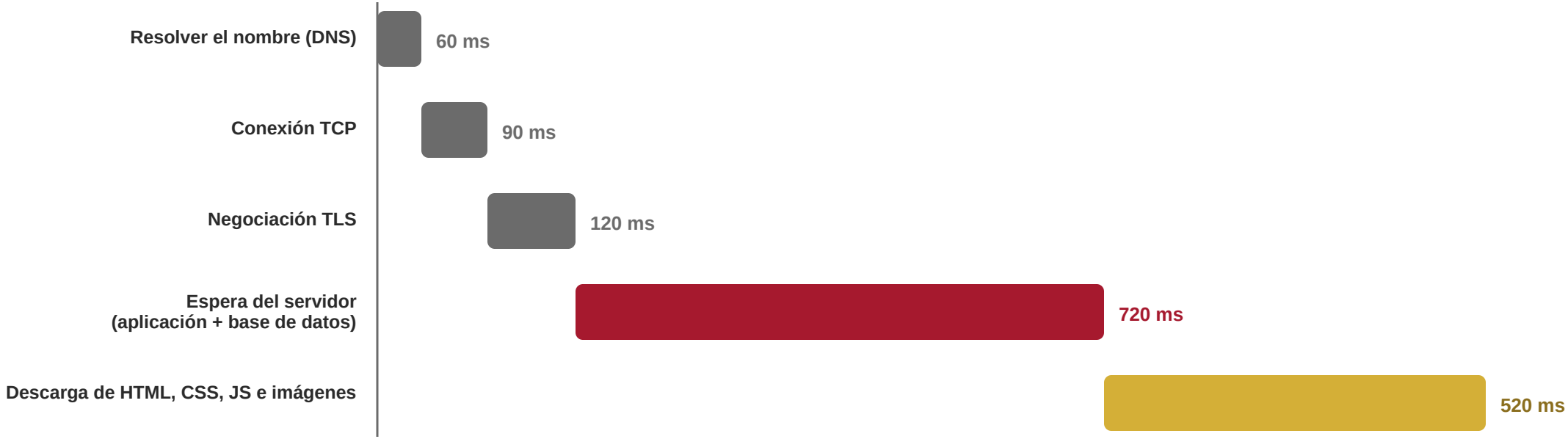
Las rutas reales son más largas que la línea recta, por eso lo medido suele superar estos mínimos teóricos.

Fuente: cálculo propio (distancia de círculo máximo, 2/3 de la velocidad de la luz) · valores teóricos

¿Dónde se va el tiempo de una página?

La espera del servidor y la descarga dominan, y ambas se pueden evitar con una copia cercana.

Una visita desde Ciudad de México, fase por fase (ms, cifras ilustrativas)



Quién puede acortar cada fase

DNS y TLS más cerca	Copias en el borde	Caché de aplicación	Copias en el navegador
Parte 3 y Parte 2	CDN · Parte 2	ElastiCache · Parte 4	TTL y cabeceras · Parte 1

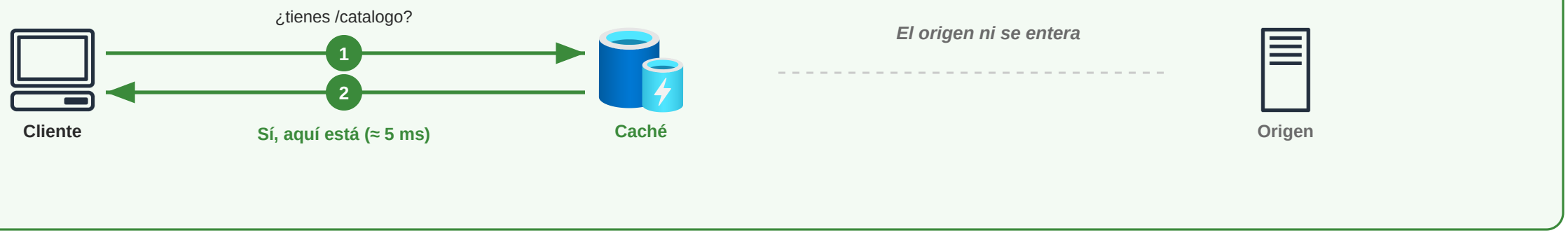
Fuente: adaptado de web.dev (Performance) y herramientas de red de navegadores · tiempos ilustrativos

CONCEPTO

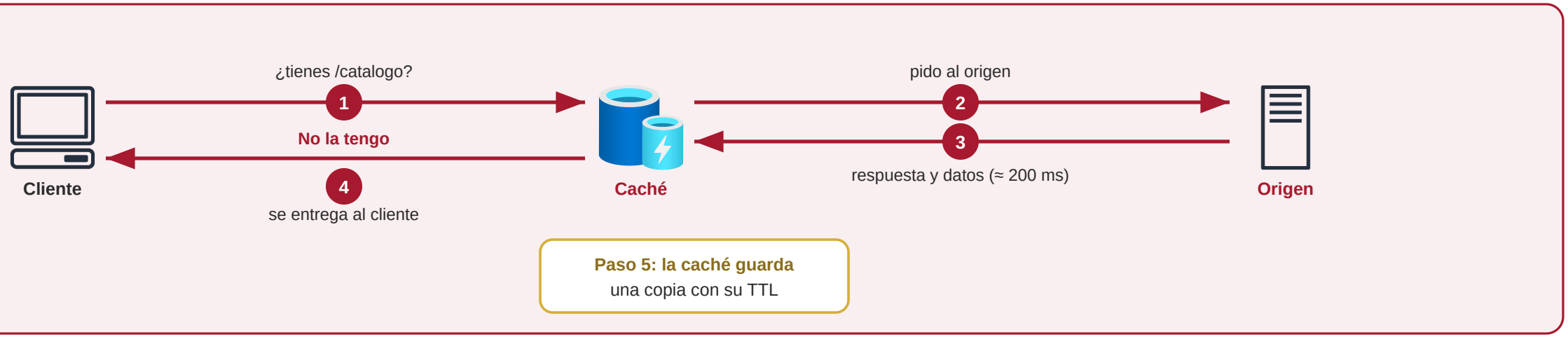
Acierto y fallo: la idea central de toda caché

Si la copia existe y está vigente se responde en milisegundos; si no, se paga el viaje una vez y se guarda.

Acierto (cache hit)



Fallo (cache miss)

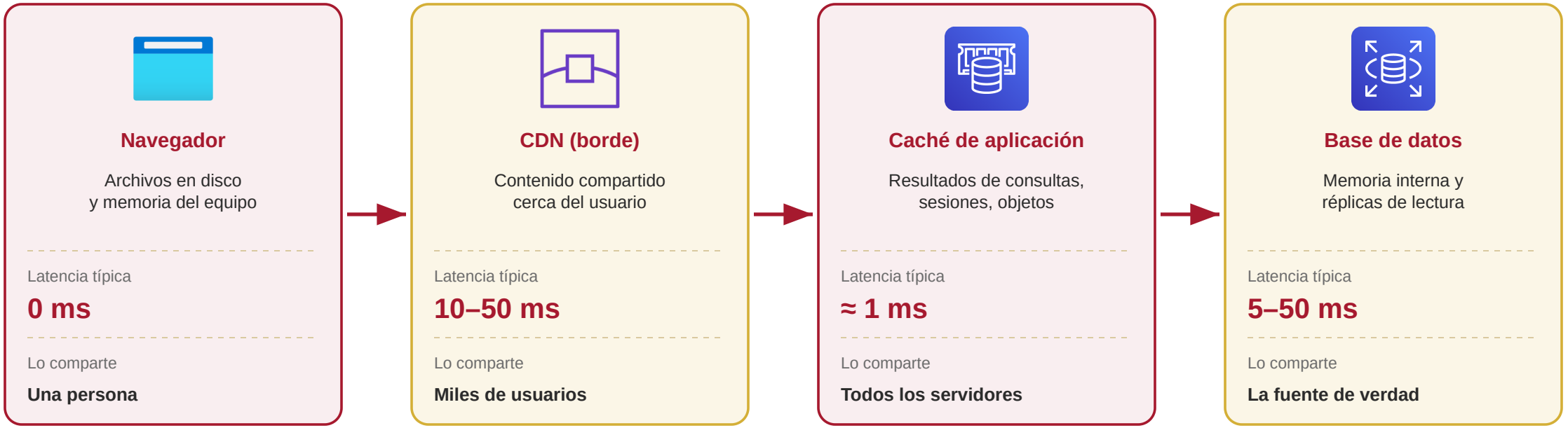


Fuente: adaptado de AWS Well-Architected, pilar Eficiencia del rendimiento · íconos AWS y Azure

CONCEPTO

La jerarquía: cuatro niveles de caché

Cuanto más cerca del usuario, más rápida es la respuesta; cuanto más abajo, más fresca y más compartida.



Si no acierta aquí, baja al siguiente nivel

Ejemplo: 100 solicitudes de una página del catálogo (ilustrativo)

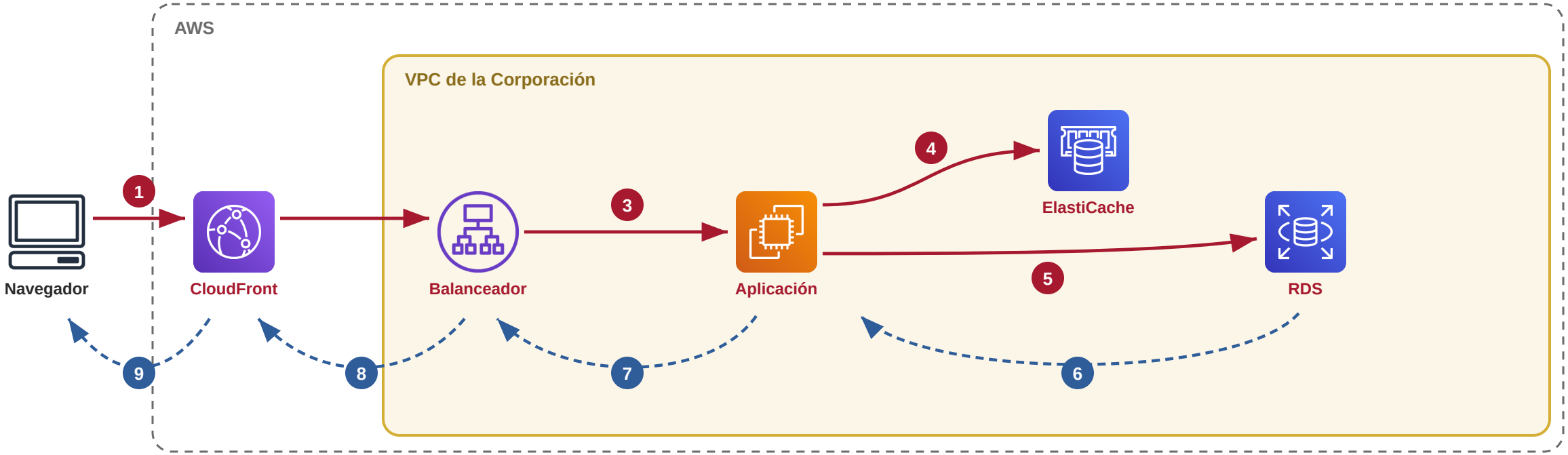


Fuente: adaptado de AWS Well-Architected (Eficiencia del rendimiento) y AWS Caching Best Practices · cifras ilustrativas

FLUJO

Camino de una petición cuando nadie tiene la copia

La primera visita recorre todos los niveles; las siguientes se detienen en el primero que acierte.

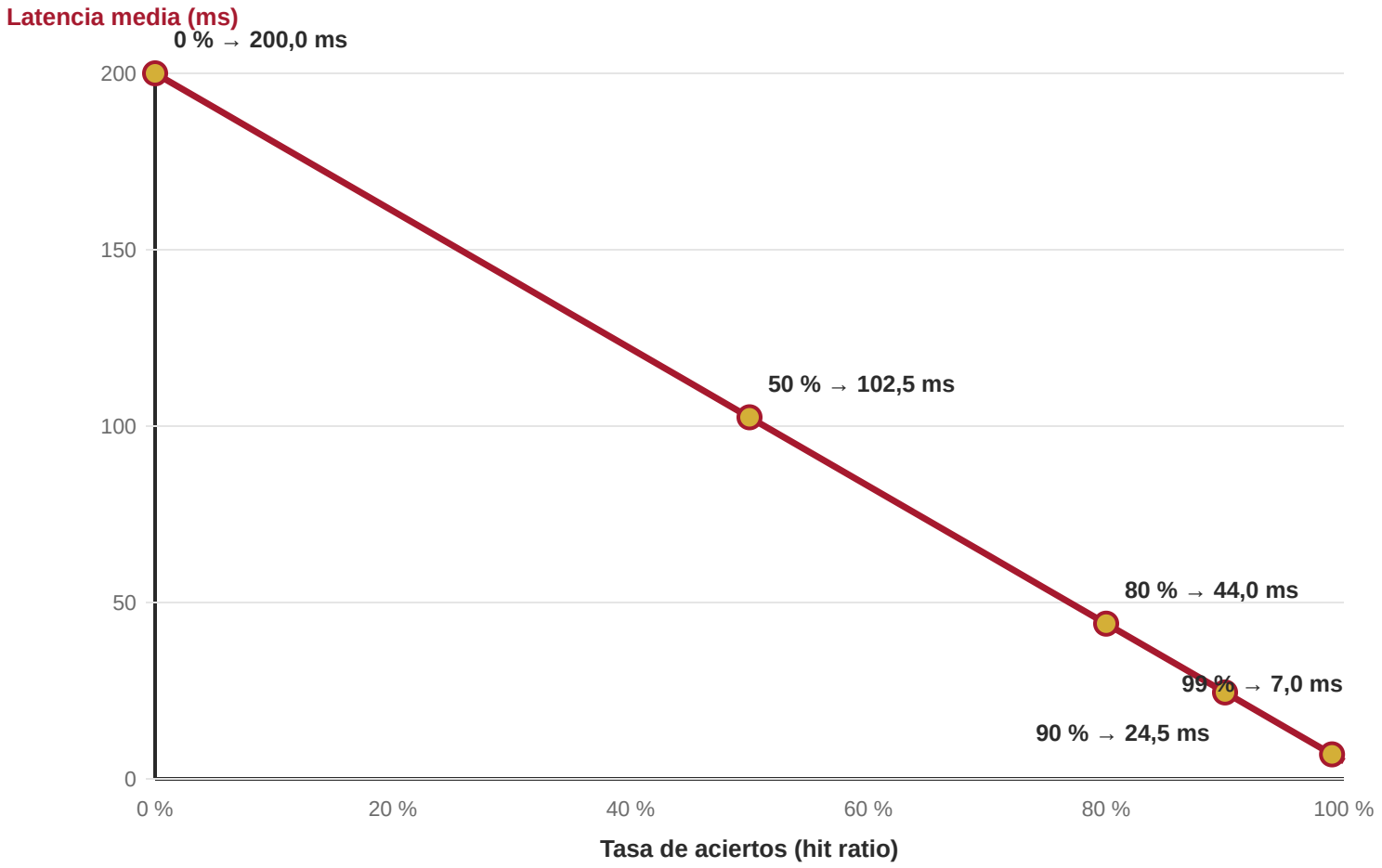


1	El navegador mira su caché; si no hay copia, pide a CloudFront
2	CloudFront no la tiene: reenvía al balanceador
3	El balanceador elige un servidor de aplicación
4	La aplicación pregunta a ElastiCache; falla
5	Entonces consulta la base de datos RDS
6–9	La respuesta vuelve y cada nivel guarda su copia

Fuente: adaptado de AWS Architecture Center · patrón web con CDN y caché · íconos AWS

La tasa de aciertos manda el rendimiento

Pasar de 90 a 99 por ciento de aciertos reduce la latencia media más de tres veces.



La fórmula

t medio = h × t caché + (1 − h) × t origen

con t caché = 5 ms, t origen = 200 ms

(h es la fracción de aciertos)

Valores ilustrativos

De 50 a 90 por ciento

baja de 102,5 a 24,5 ms:
la mitad de aciertos no basta.

De 90 a 99 por ciento

baja de 24,5 a 7 ms: el último
tramo cuesta más pero rinde.

Mide CacheHitRate: sin ese número no sabes si tu caché trabaja.

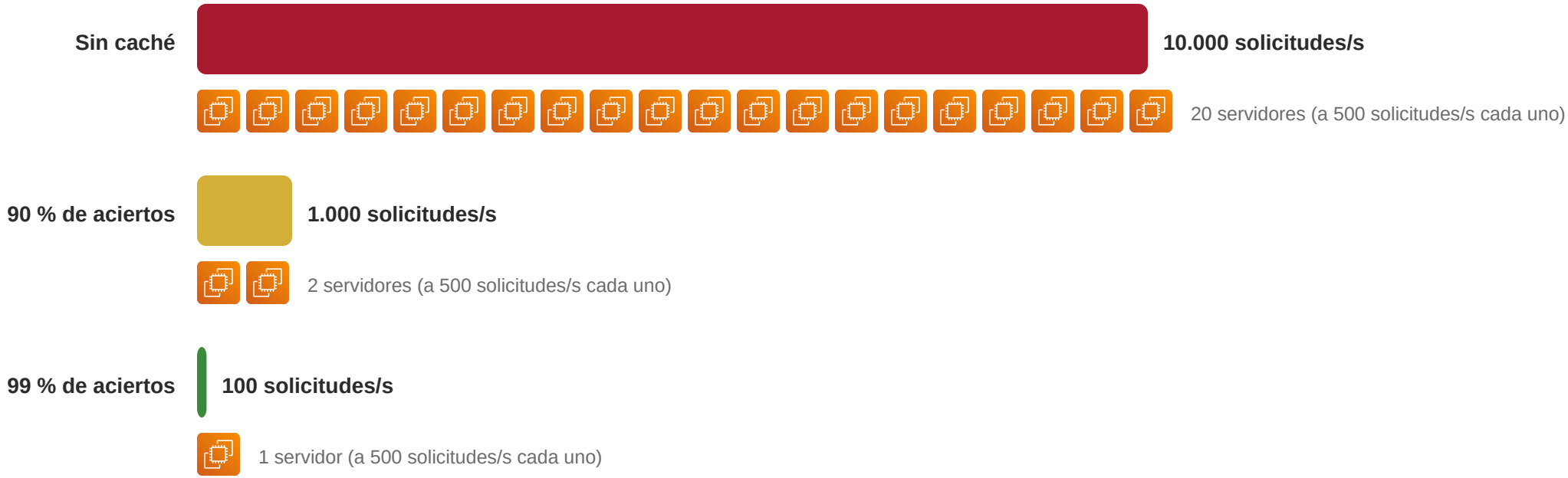
Fuente: modelo propio de latencia efectiva · documentación de métricas de CloudFront y ElastiCache · valores ilustrativos

IMPACTO

Menos carga en el origen, menos servidores

Cada acierto es una solicitud que el origen no atiende: la caché es capacidad que no se compra.

Pico de 10.000 solicitudes por segundo: ¿cuántas llegan al origen y cuántos servidores hacen falta?



Lectura para el arquitecto

Una caché es una forma de escalar: absorbe lecturas repetidas y deja a la base de datos las escrituras y las consultas únicas.

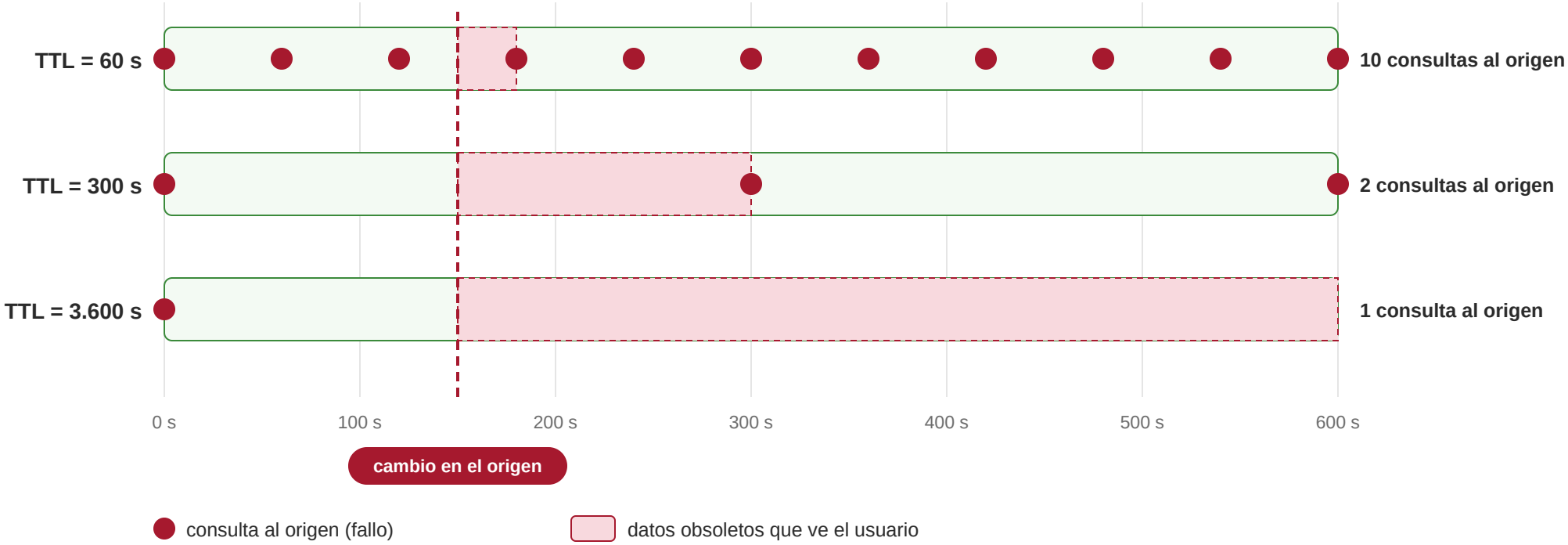
También amortigua picos (campañas, cierres de mes) y reduce el costo de cómputo y de transferencia desde el origen.

Fuente: adaptado de AWS Well-Architected (Eficiencia del rendimiento y Optimización de costos) · cálculo ilustrativo

TTL: el equilibrio entre frescura y rendimiento

Un TTL corto protege la frescura; uno largo protege al origen. No existe un valor correcto para todo.

Una ventana de 10 minutos; el contenido cambia en el origen a los 150 s



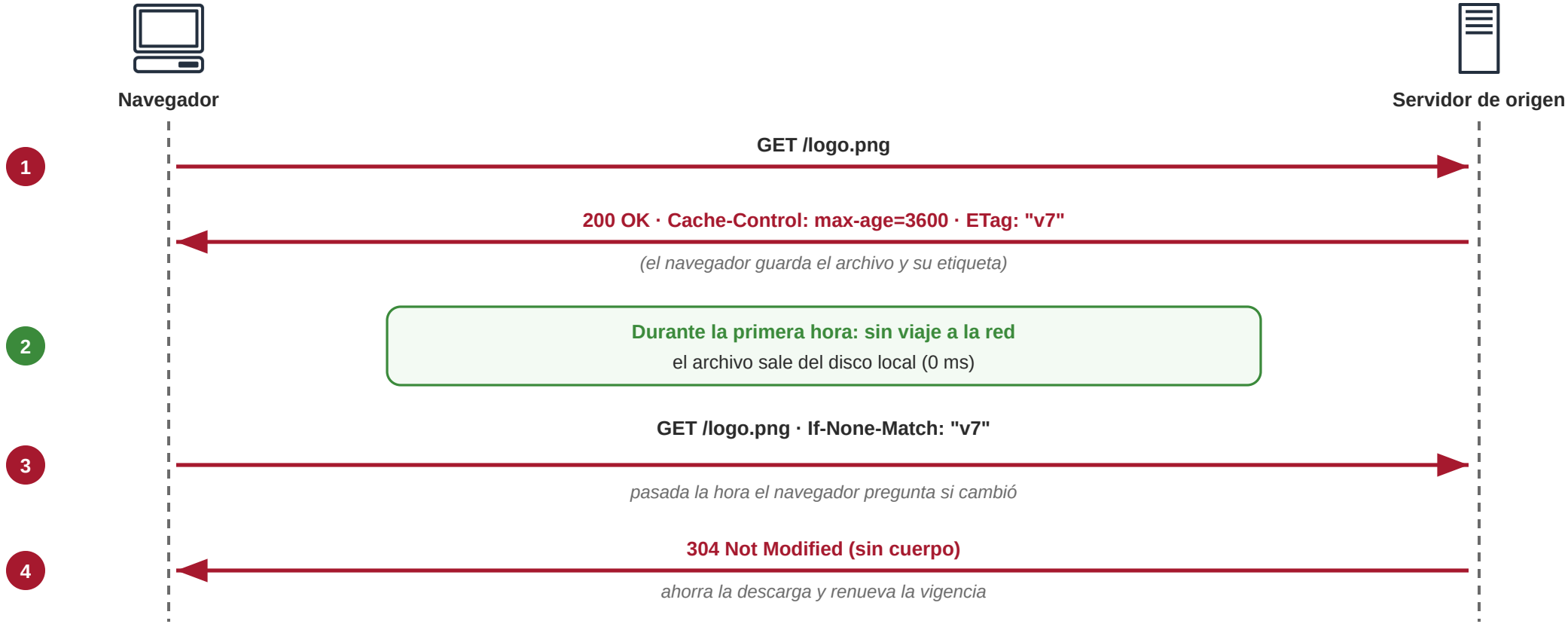
TTL (time to live): cuánto tiempo se considera vigente una copia.

TTL corto = datos frescos pero menos aciertos; TTL largo = más aciertos pero más tiempo de obsolescencia. Se elige por tipo de dato.

Fuente: adaptado de MDN Web Docs (HTTP caching) y AWS CloudFront Developer Guide · simulación ilustrativa

Las cabeceras HTTP dicen cómo se cachea

El origen controla la caché con Cache-Control y valida copias con ETag, sin reenviar el archivo completo.



Cache-Control: public / private / no-store / max-age / s-maxage

public permite cachés compartidas (CDN); private solo el navegador; no-store prohíbe guardar; s-maxage fija el TTL de las cachés compartidas.

Fuente: adaptado de MDN Web Docs — HTTP caching y RFC 9111 · íconos AWS

Qué se puede cachear: del archivo estático al dato vivo

Cuanto más igual sea para todos y menos cambie, más tiempo puede vivir en caché.

Estático · idéntico para todos → **Dinámico · por usuario o transaccional**



Logo, CSS y JS con versión

TTL sugerido

1 año

Archivos con huella en el nombre (app.v42.js)



Imágenes y fichas técnicas

TTL sugerido

días

Cambian poco; se versionan al actualizarse



Listados y fichas de producto

TTL sugerido

minutos

Iguales para todos, cambian a diario



Precios y existencias

TTL sugerido

segundos

Muy cambiantes; TTL cortísimo o evento



Carrito y sesión

TTL sugerido

solo privado

Cache-Control: private; en el navegador o en Redis



Pagos, saldo y pedidos

TTL sugerido

no cachear

no-store: siempre se consulta la fuente

Criterio práctico: ¿es igual para todos y tolera unos segundos de retraso? Entonces se cachea.

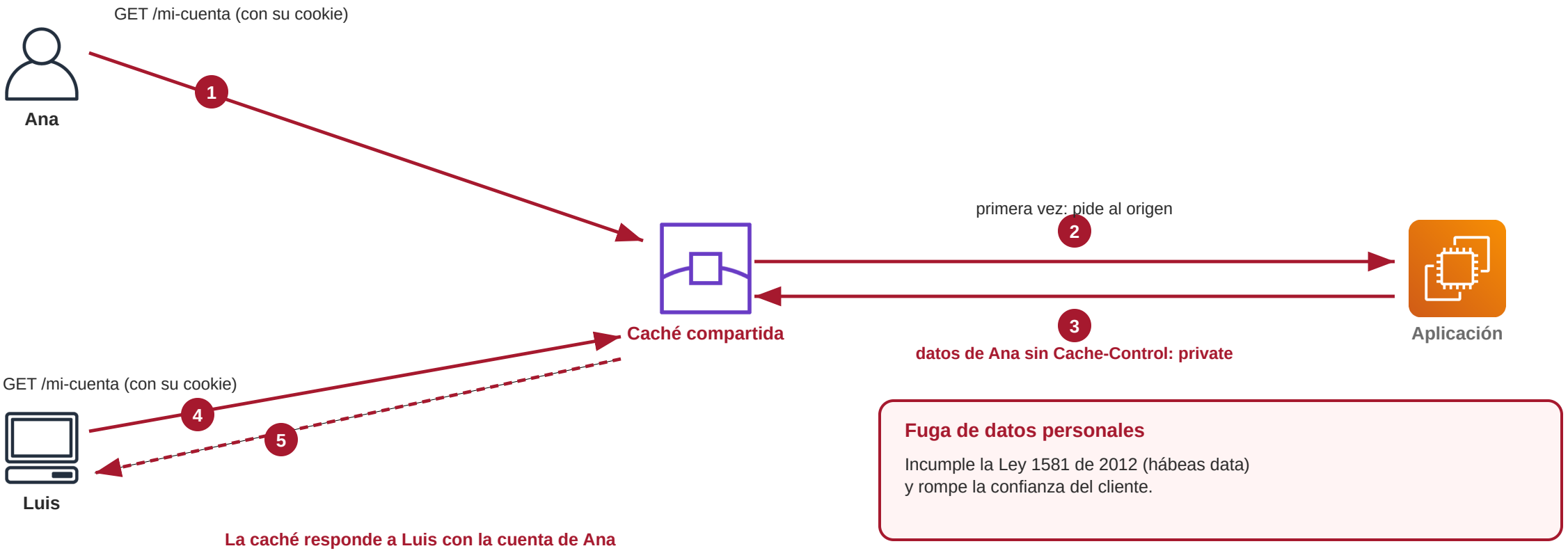
Si cambia por usuario o mueve dinero, no pasa por una caché compartida; a lo sumo, por una privada con clave de usuario.

Fuente: adaptado de AWS CloudFront Developer Guide y MDN Web Docs · TTL orientativos

QUÉ NO CACHEAR

Lo personal no va a una caché compartida

Una respuesta de usuario guardada por error en una caché compartida se entrega a la persona equivocada.



Fuga de datos personales

Incumple la Ley 1581 de 2012 (hábeas data) y rompe la confianza del cliente.

Prevención: Cache-Control: private o no-store en respuestas personales · rutas separadas (/api/privado/*) sin caché en la CDN

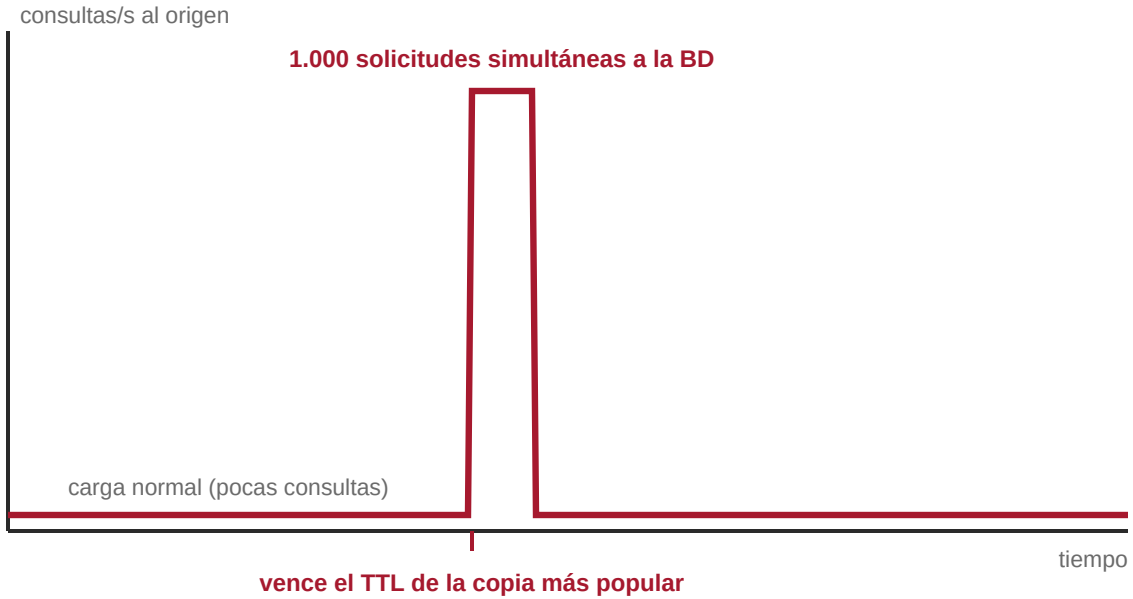
Si hay que cachear por usuario, la clave de caché debe incluir la identidad (cabecera Vary o ID de sesión) y revisarse con pruebas.

Fuente: adaptado de MDN Web Docs (HTTP caching) y OWASP Web Cache Deception · Ley 1581 de 2012

Tres problemas clásicos de las cachés

La estampida ocurre cuando una copia popular vence y todos van al origen a la vez; se previene con diseño.

Estampida de caché (cache stampede)



Otros dos problemas clásicos

Datos obsoletos: el usuario ve un precio viejo porque la copia aún no vence.

Invalidación difícil: borrar justo lo que cambió sin vaciar toda la caché (arranque en frío).

Cómo se evita

1. **TTL con variación aleatoria (jitter)**
   Cada copia vence en un momento distinto
2. **Coalescencia de solicitudes**
   Un solo viaje al origen; los demás esperan
3. **Servir contenido caducado mientras se renueva**
   stale-while-revalidate / stale-if-error
4. **Bloqueo breve al recalcular**
   Un proceso reconstruye, los demás usan el valor previo

Fuente: adaptado de AWS Builders Library (Caching challenges and strategies) y RFC 5861 · ilustrativo

CIERRE

Lo que queda de la Parte 1

Cachear es acercar copias al usuario respetando frescura y privacidad; la CDN es el siguiente escalón.



Navegador

Cache-Control y ETag: copias en el equipo

1



CDN

Copias compartidas cerca del usuario

2



Aplicación

Resultados y sesiones en memoria

3



Base de datos

Última línea; solo lo que nadie tiene

4

Para el portal de la Corporación Industrial del Caribe

Meta: 90 % de aciertos o más en imágenes y documentos, TTL de 1 año con versión para CSS y JS, y cabeceras private/no-store en todo lo que sea de un cliente concreto.

Ideas clave

- La distancia fija la latencia
- Hit ratio = rendimiento
- TTL equilibra frescura
- Lo personal no se comparte

Siguiente: Parte 2

Redes de distribución de contenido con CloudFront

Fuente: síntesis de la Parte 1 · AWS Well-Architected, pilar Eficiencia del rendimiento

SEMANA 12 · CACHÉ Y DISTRIBUCIÓN DE CONTENIDO

Redes de distribución de contenido (CDN)

Parte 2 de 4 · Unidad 3

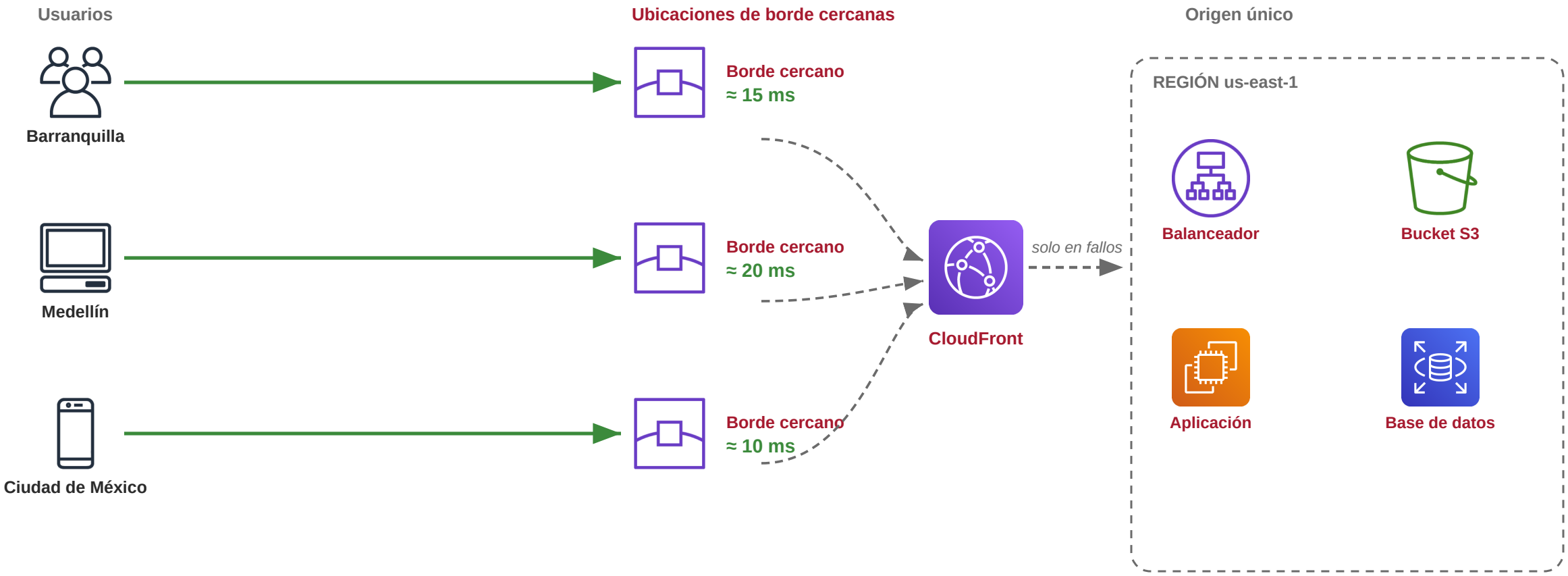
Ing. Rodolfo Cañas Cervantes
Universidad de la Costa (CUC) · Ingeniería de Sistemas
Semana 12 · Periodo 2026-2 · Barranquilla, Colombia

Ubicaciones de borde, orígenes, caché, HTTPS y protección: cómo funciona CloudFront.

ENCUADRE

Una CDN acerca el contenido al usuario

Cada usuario habla con el borde más cercano; el origen solo atiende lo que no está en copia.



Catálogo desde Ciudad de México: 4,8 s con un solo origen → 1,3 s con CDN (ilustrativo)

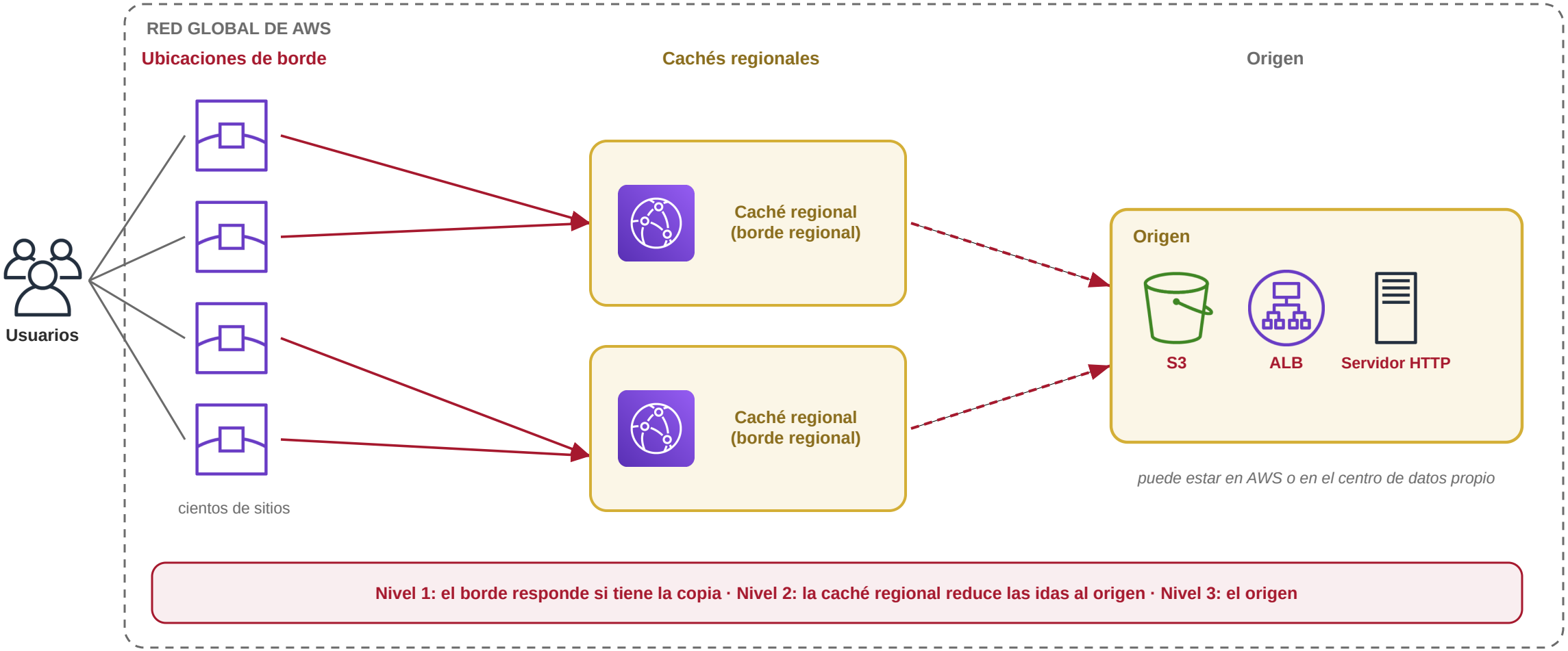
Los estáticos y muchas respuestas dinámicas salen del borde; el origen atiende solo lo que ninguna copia tiene.

Fuente: adaptado de AWS CloudFront Developer Guide (How CloudFront delivers content) · tiempos ilustrativos

ANATOMÍA

Borde, caché regional y origen: tres niveles

Un fallo en el borde se intenta primero en la caché regional antes de molestar al origen.

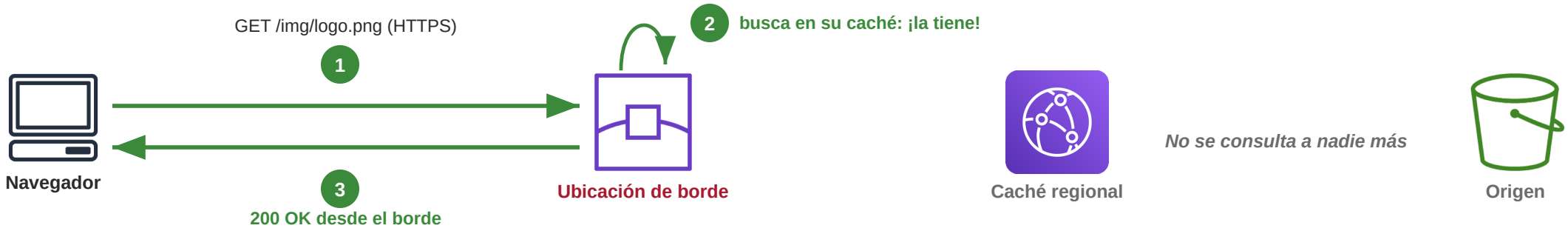


Fuente: adaptado de AWS CloudFront Developer Guide (Regional edge caches) · íconos AWS

FLUJO

Petición con acierto: la copia está en el borde

El borde responde con la copia vigente y el origen no recibe ninguna solicitud.



Tiempo de respuesta (ilustrativo)

Acierto en el borde	≈ 20 ms	
Fallo, desde el origen		≈ 250 ms (siguiente diapositiva)

Cabeceras de respuesta que lo confirman

X-Cache: Hit from cloudfront

Age: 1830 (segundos que la copia lleva en caché)

X-Amz-Cf-Pop: BOG51 (ubicación de borde que respondió; ejemplo)

Útiles para depurar con curl -I o con las herramientas del navegador.

Lo que ganó la Corporación

≈ 20 ms de respuesta en lugar de ≈ 250 ms desde el origen en Virginia

Sin consumir CPU del servidor ni ancho de banda de la región.

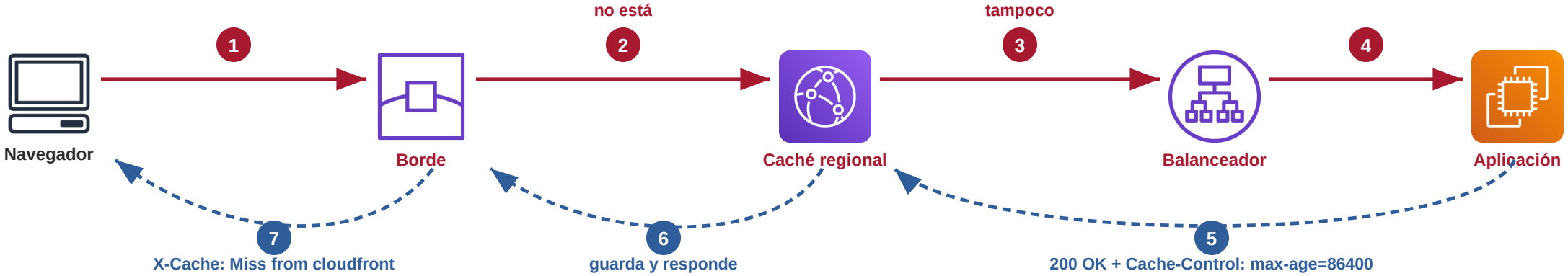
El origen ni se entera de la visita.

Fuente: adaptado de AWS CloudFront Developer Guide (Understanding the request flow) · valores ilustrativos

FLUJO

Petición con fallo: del borde al origen y de vuelta

El primer usuario paga el viaje completo; los demás reciben la copia que quedó guardada.



¿Cuánto cuesta el fallo? (≈ 250 ms en total, ilustrativo)



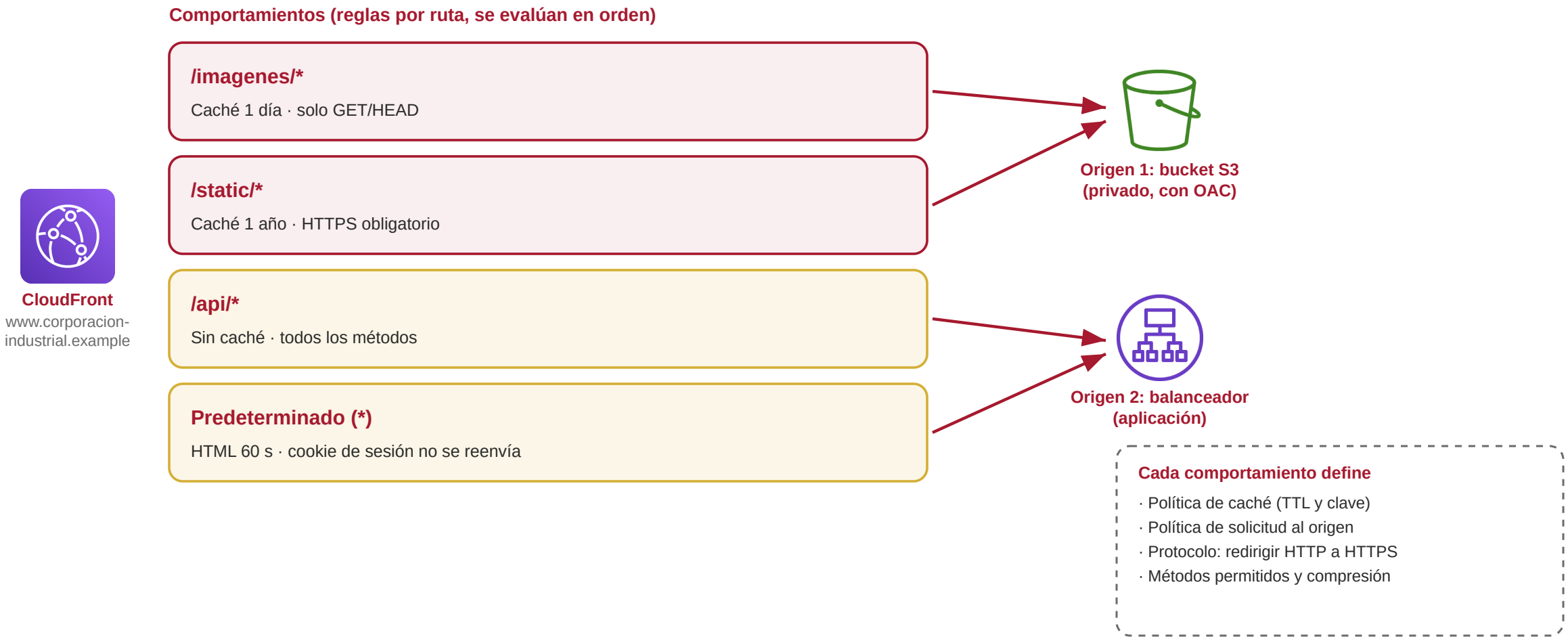
Qué pasa durante un fallo

- Cada nivel guarda la copia al regresar: la próxima visita será un acierto.
- Si 100 usuarios piden el mismo objeto a la vez, CloudFront agrupa las solicitudes y envía solo una al origen.
- Un fallo cuesta más que consultar directo al origen (un salto extra), pero ocurre una vez por objeto y ubicación.
- Respuestas de error (404, 5xx) también se pueden cachear unos segundos con TTL de error.

Fuente: adaptado de AWS CloudFront Developer Guide (Understanding the request flow) · íconos AWS

Orígenes y comportamientos: quién responde a cada ruta

Una distribución puede combinar varios orígenes y aplicar una política de caché distinta por ruta.



Fuente: adaptado de AWS CloudFront Developer Guide (Cache behavior settings) · dominio de ejemplo

La clave de caché decide si hay acierto

Cada combinación distinta de URL, parámetros, cabeceras o cookies es una copia diferente.

Tres solicitudes casi idénticas

/catalogo?id=5&utm_source=correo

/catalogo?id=5&utm_source=anuncio

/catalogo?id=5&utm_source=redes

→

Clave de caché mínima

Incluye:

· ruta /catalogo
· parámetro id

Ignora:

· utm_source y otros de seguimiento
· cookies de analítica

→



1 fallo + 2 aciertos

una sola copia en caché

tasa de aciertos: 67 % con 3 solicitudes y 99 % cuando crece el tráfico

Si la clave incluye todo (todas las cadenas de consulta, cookies y cabeceras)

copia 1: utm_source distinto

copia 2: utm_source distinto

copia 3: utm_source distinto

3 fallos · 3 copias · el origen sufre

Cada variación crea una entrada nueva: la tasa de aciertos se desploma y la caché se llena de duplicados.

Regla de oro: la clave de caché solo incluye lo que cambia la respuesta.

Reenvía al origen solo lo necesario (por ejemplo, Accept-Language si el contenido cambia por idioma).

Con las políticas administradas de CloudFront (CachingOptimized, CachingDisabled) se parte de valores seguros.

Fuente: adaptado de AWS CloudFront Developer Guide (Understanding the cache key) · ejemplo ilustrativo

TTL en CloudFront: mínimo, predeterminado y máximo

La cabecera Cache-Control del origen manda, siempre dentro del rango que fija la política.

¿Cuánto vive el objeto en el borde?



1 El origen dice max-age=3600

Está entre el mínimo y el máximo

Se cachea 3.600 s (1 hora)

2 El origen no envía cabecera de caché

Se aplica el valor predeterminado

Se cachea 86.400 s (24 h)

3 El origen dice max-age=63072000 (2 años)

Supera el TTL máximo

Se recorta a 31.536.000 s (1 año)

Políticas administradas

CachingOptimized: mín. 1 s · pred. 24 h · máx. 1 año
CachingDisabled: todos los TTL en 0 (para APIs y datos privados)

Atajo práctico

Controla el TTL desde el origen con Cache-Control: CloudFront lo respeta dentro del rango. s-maxage solo aplica a cachés compartidas como CloudFront.

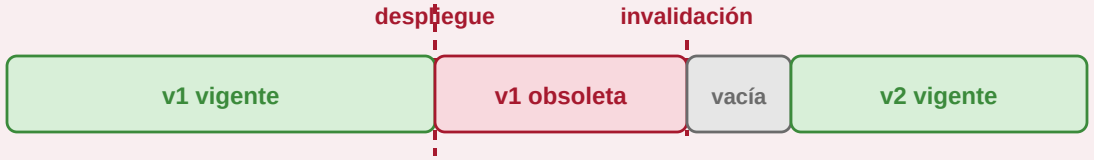
Fuente: adaptado de AWS CloudFront Developer Guide (Managing how long content stays in the cache) · políticas administradas

Invalidar o versionar: cómo renovar el contenido

La forma más limpia de actualizar un archivo en la CDN es darle un nombre nuevo.

Opción A · Invalidar la ruta

Copia en la ubicación de borde a lo largo del tiempo

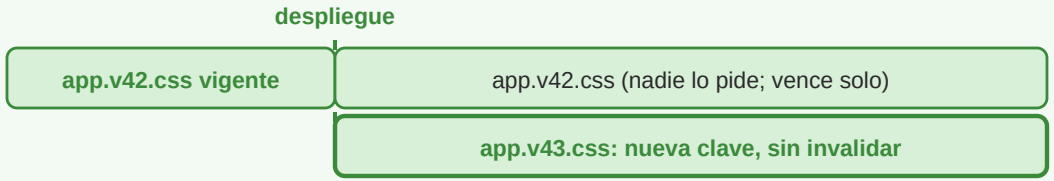


1. Despliegas el CSS nuevo con el mismo nombre
2. Creas la invalidación de /css/app.css (o /css/*)
3. Las ubicaciones eliminan la copia en poco tiempo
4. La siguiente solicitud es un fallo y trae la nueva

Cuesta tiempo de propagación; las primeras 1.000 rutas al mes son gratuitas (verifica la tarifa vigente).

Opción B · Cambiar el nombre (versionado)

Copia en la ubicación de borde a lo largo del tiempo



1. Cada versión tiene un nombre distinto
2. index.html (TTL corto) apunta a la v43
3. La v43 es un fallo la primera vez; la v42 envejece sola
4. Sin invalidaciones y con TTL de un año

Es la práctica recomendada para CSS, JS e imágenes; la invalidación queda para emergencias.

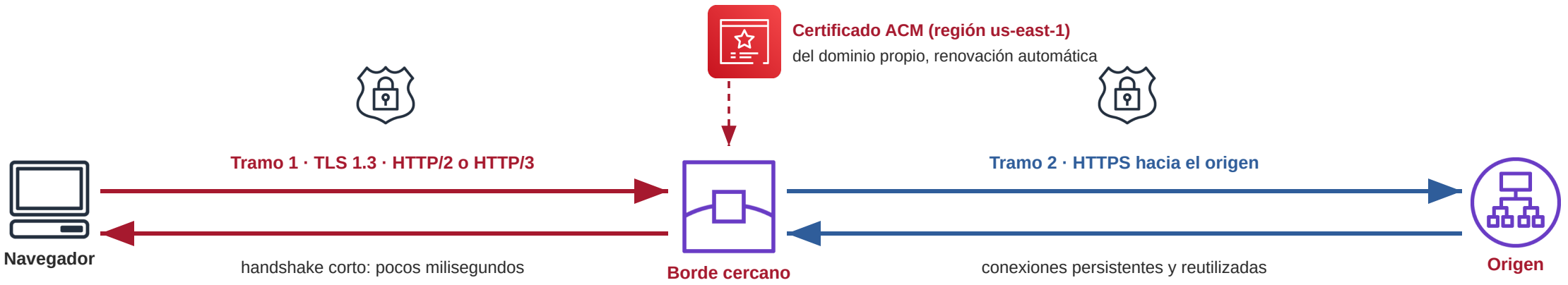
Y el HTML principal

Dale un TTL corto (por ejemplo 60 s) y revisa que los enlaces a los archivos versionados lleguen en cada despliegue (CI/CD).

Fuente: adaptado de AWS CloudFront Developer Guide (Invalidating files) · tarifas sujetas a cambio

HTTPS en el borde: dos tramos cifrados

El usuario negocia TLS con el borde cercano y el borde mantiene conexiones seguras con el origen.



Resultado: la negociación TLS ocurre a pocos milisegundos del usuario y el origen casi nunca paga un handshake completo

Por qué el TLS en el borde

· Negociar TLS con un servidor a 15 ms en lugar de uno a 150 ms
· La CDN absorbe el costo criptográfico
· Un solo lugar para certificados

Certificados

· ACM emite y renueva certificados públicos sin costo para servicios integrados
· Para CloudFront deben estar en us-east-1
· Dominio propio: www.corporacion-industrial.example (un CNAME o alias)

Configura siempre

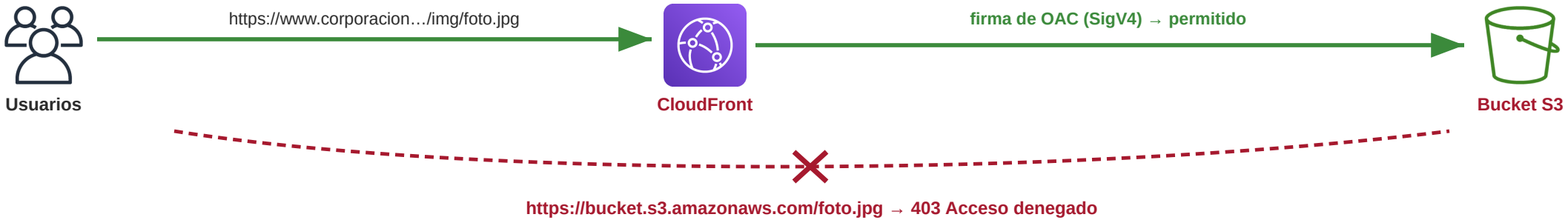
· Política del visitante: redirigir HTTP a HTTPS
· Política hacia el origen: solo HTTPS
· Política de seguridad TLS mínima actual (TLS 1.2 o superior)
· HSTS mediante cabeceras de respuesta

Fuente: adaptado de AWS CloudFront Developer Guide (Using HTTPS with CloudFront) y documentación de ACM

Protege el origen: solo CloudFront puede entrar

Si el origen sigue abierto a Internet, un atacante rodea la CDN y su WAF; hay que cerrarle esa puerta.

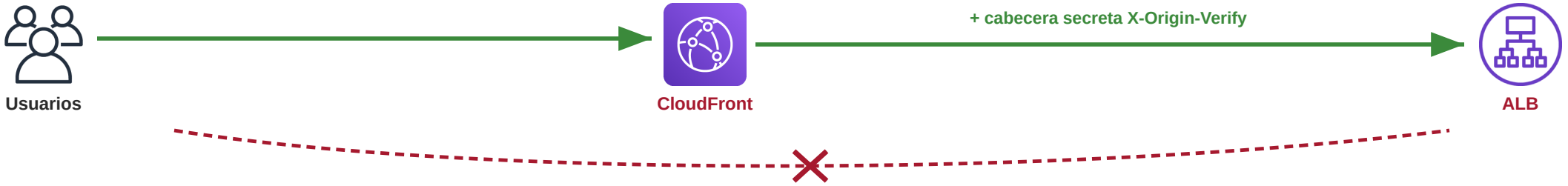
Bucket S3 privado con Origin Access Control (OAC)



Política del bucket: permite s3:GetObject solo al servicio cloudfront.amazonaws.com cuando el origen es esa distribución (condición AWS:SourceArn).

OAC reemplaza al antiguo OAI y admite SSE-KMS y todas las regiones.

Balanceador (ALB) solo accesible desde CloudFront

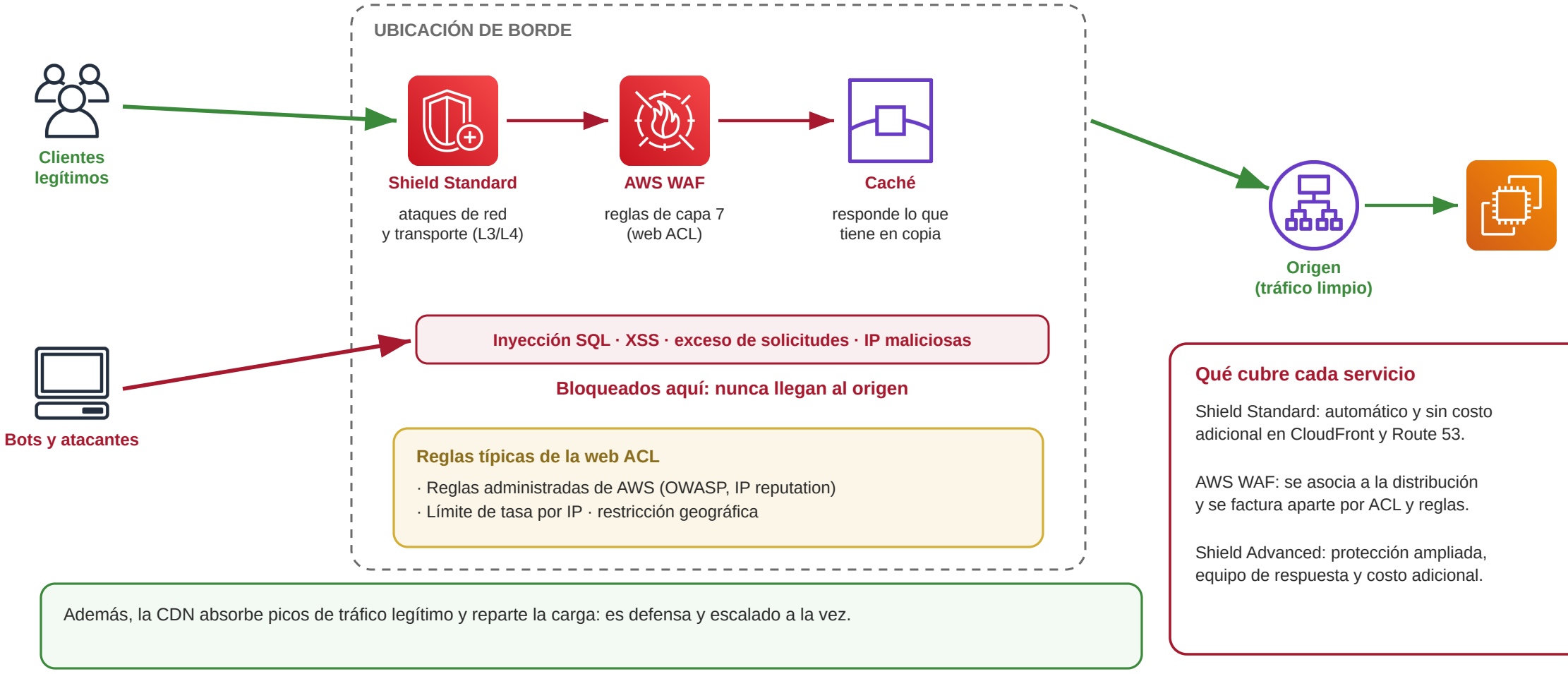


Grupo de seguridad del ALB: entrada 443 solo desde la lista de prefijos administrada com.amazonaws.global.cloudfront.origin-facing.

Fuente: adaptado de AWS CloudFront Developer Guide (Restricting access to an Amazon S3 origin / to Application Load Balancers)

El borde también filtra: Shield y WAF

Lo malicioso se detiene en la ubicación de borde, antes de gastar recursos del origen.



Además, la CDN absorbe picos de tráfico legítimo y reparte la carga: es defensa y escalado a la vez.

Qué cubre cada servicio

Shield Standard: automático y sin costo adicional en CloudFront y Route 53.

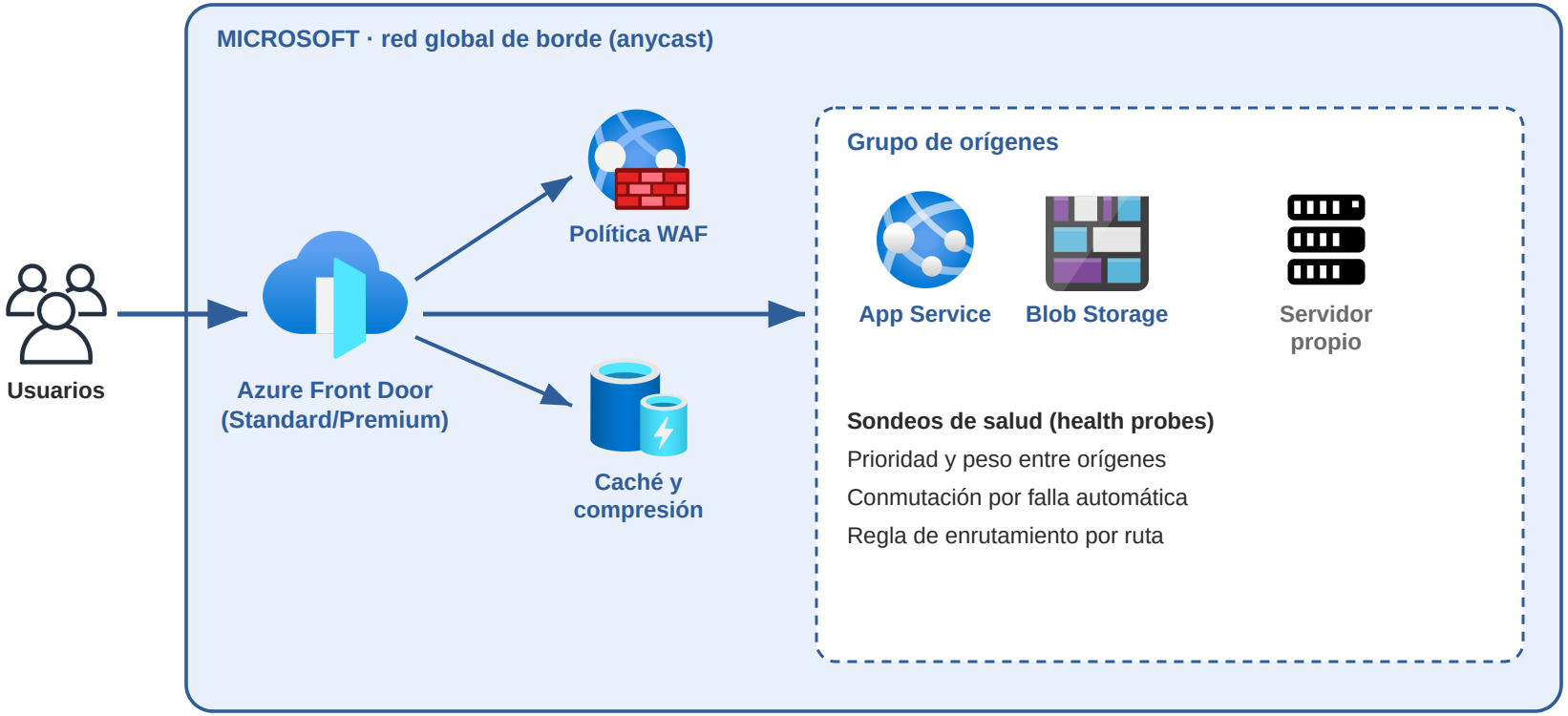
AWS WAF: se asocia a la distribución y se factura aparte por ACL y reglas.

Shield Advanced: protección ampliada, equipo de respuesta y costo adicional.

Fuente: adaptado de AWS WAF Developer Guide y AWS Shield (documentación) · íconos AWS

Otro ejemplo: Azure Front Door y Cloud CDN

Aunque cambien los nombres, toda CDN combina borde, caché, orígenes protegidos, TLS y WAF.



Google Cloud CDN

se activa sobre el balanceador HTTP(S) externo

Mismos conceptos

- Borde cercano con caché
- Origen (uno o varios)
- Regla por ruta y TTL
- Invalidación de caché
- HTTPS y WAF en el borde

Idea para llevar

El diseño es el mismo en todas las nubes: borde, caché, origen protegido, TLS y filtro. Cambian los nombres y los menús.

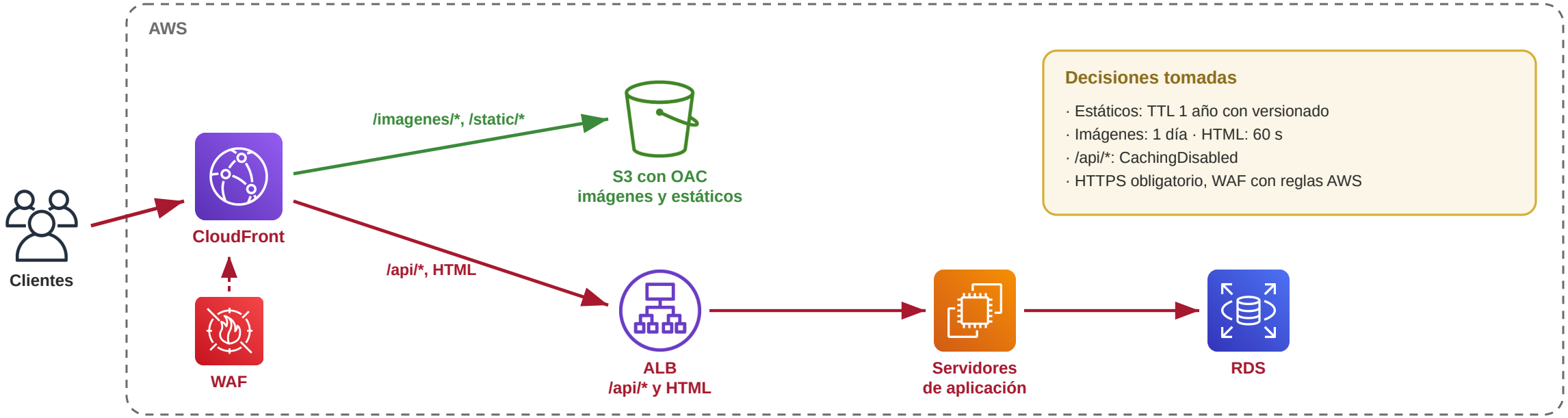
En el curso practicamos con CloudFront porque es lo que ofrece AWS Academy.

Fuente: adaptado de Microsoft Learn (Azure Front Door) y Google Cloud (Cloud CDN) · íconos Azure y GCP

CASO APLICADO

Corporación Industrial: CloudFront delante del portal

Estáticos desde S3 por OAC, dinámico desde el balanceador y un WAF delante de todo.



4,8 s → 1,3 s
catálogo desde Ciudad de México

≈ 90 %
de solicitudes resueltas en el borde

−85 %
de carga en servidores y base de datos

Resultados esperados e ilustrativos: se verifican con CloudWatch (CacheHitRate, 4xxErrorRate, Requests) y con pruebas desde varias ciudades.

Fuente: caso docente Corporación Industrial del Caribe · adaptado de AWS Architecture Center · cifras ilustrativas

CIERRE

Lo que queda de la Parte 2

Una CDN bien configurada acelera, ahorra y protege al origen; el DNS es el siguiente paso.



Borde y caché regional

Copias cerca del usuario, en dos niveles.

1



Comportamientos y clave

Reglas por ruta; la clave solo con lo que cambia.

2



HTTPS, OAC y WAF

TLS en el borde y el origen cerrado a Internet.

3



Mismo diseño, otras nubes

Azure Front Door y Cloud CDN aplican lo mismo.

4

Pregunta para pensar antes de la Parte 3

¿Cómo llega el navegador al borde correcto? Antes de que CloudFront responda,
alguien tuvo que traducir www.corporacion-industrial.example a una dirección IP: eso es DNS.

Ideas clave

- La CDN responde en el borde
- La clave de caché define aciertos
- Versiona en vez de invalidar
- Cierra el origen a Internet

Siguiente: Parte 3

DNS a escala con Route 53:
resolución y enrutamiento

Fuente: síntesis de la Parte 2 · AWS CloudFront Developer Guide

SEMANA 12 · CACHÉ Y DISTRIBUCIÓN DE CONTENIDO

DNS a escala: resolución y enrutamiento con Route 53

Parte 3 de 4 · Unidad 3

Ing. Rodolfo Cañas Cervantes

Universidad de la Costa (CUC) · Ingeniería de Sistemas

Semana 12 · Periodo 2026-2 · Barranquilla, Colombia

Cómo un nombre se convierte en una dirección y cómo decidir a dónde enviar a cada usuario.

ENCUADRE

DNS: el primer paso de toda visita

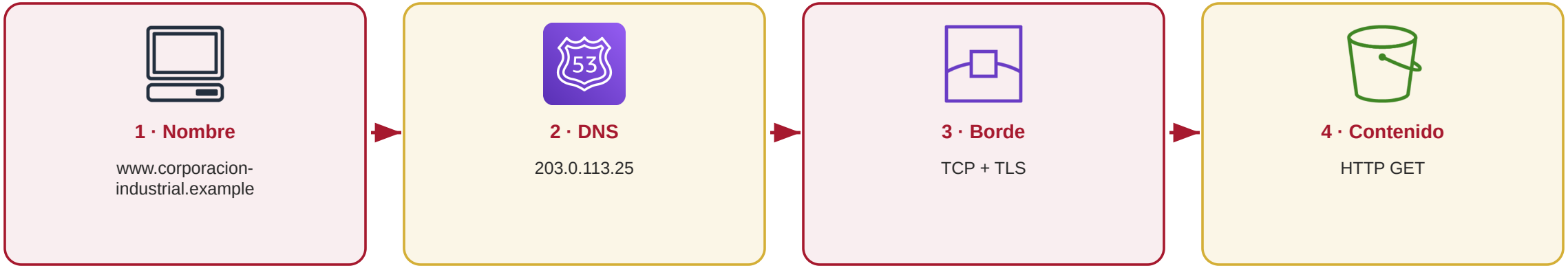
Antes de abrir una conexión, el navegador debe traducir el nombre a una dirección IP.



El DNS es la agenda de contactos de Internet

- Convierte nombres legibles en direcciones IP
- Es consulta obligada antes de abrir la conexión
- Puede responder distinto según quién pregunta

Después de la respuesta, el navegador abre TCP y TLS con esa dirección



En la cascada de la Parte 1, el DNS aparece como la primera barra: lo que no se acelera aquí retrasa todo lo demás.

Fuente: adaptado de AWS Route 53 Developer Guide (How DNS works) · direcciones de documentación RFC 5737

JERARQUÍA

El DNS es un árbol de delegaciones

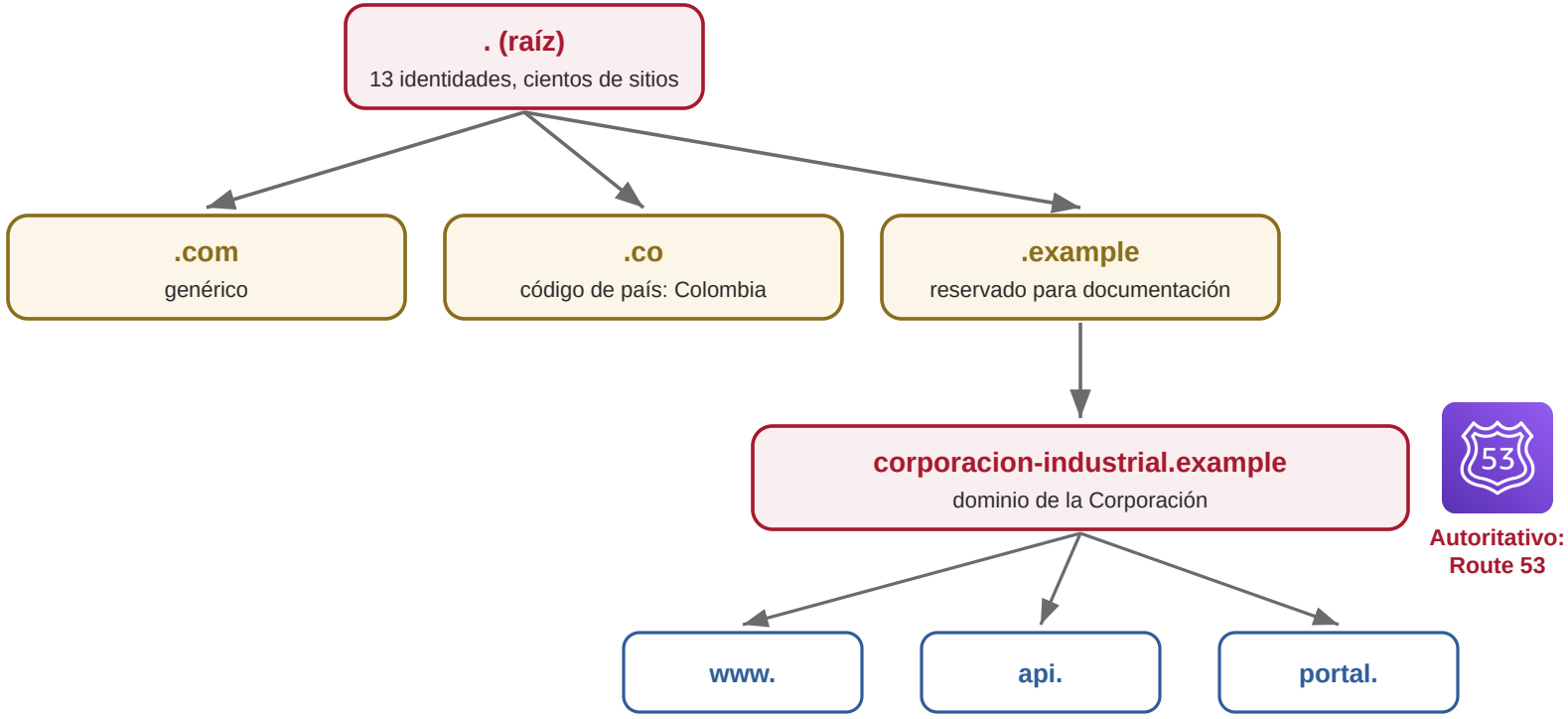
Raíz, dominios de primer nivel y servidores autoritativos: cada nivel delega al siguiente.

Quién responde en cada nivel

· Servidores raíz: señalan al TLD
· Servidores del TLD: señalan a los servidores de nombres del dominio
· Servidores autoritativos: dan la IP

Delegación

El registrador anota en el TLD cuáles son los servidores de nombres (NS) de tu zona. Con Route 53 son cuatro, repartidos en dominios distintos.



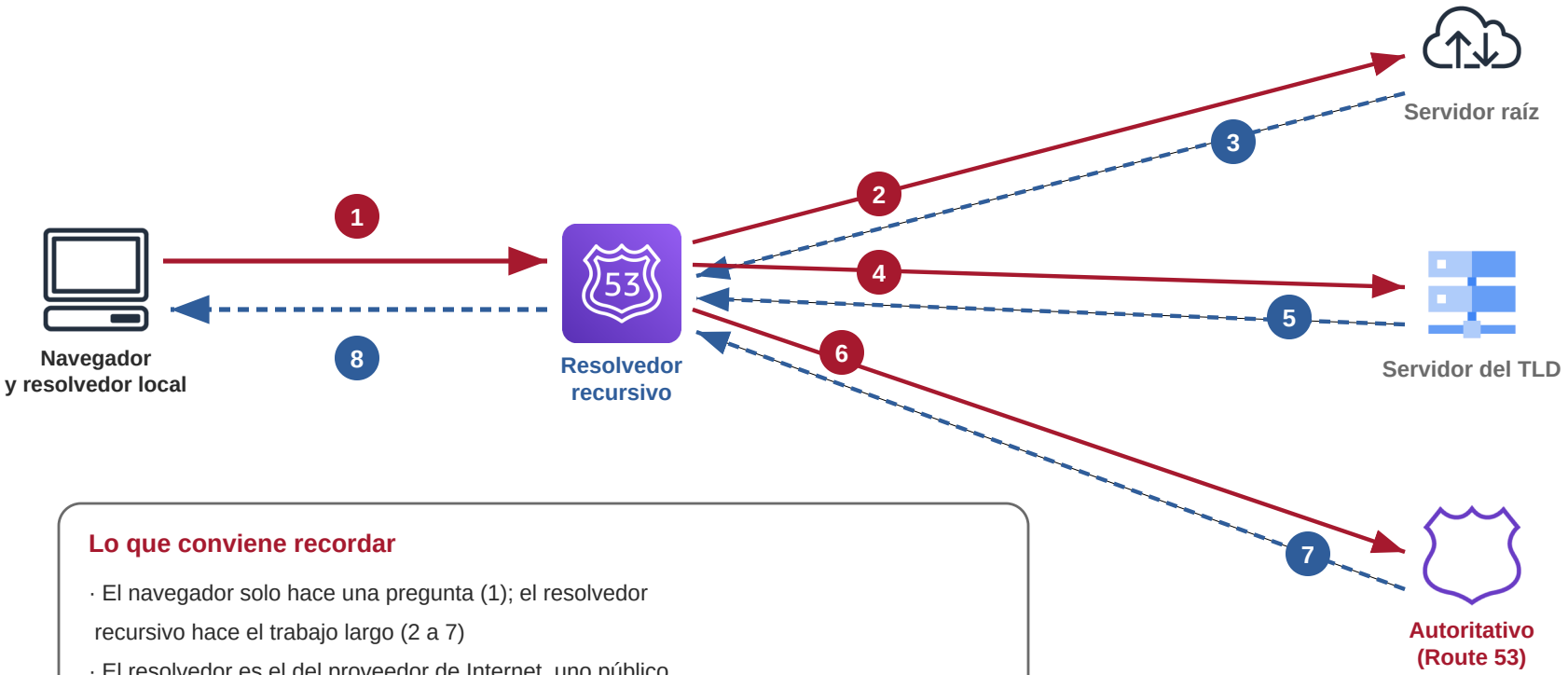
Nadie conoce todo el árbol: cada nivel solo sabe a quién preguntar después.

Por eso la resolución es una cadena de referencias y no una búsqueda central.

Fuente: adaptado de AWS Route 53 Developer Guide y RFC 1034 · dominio de ejemplo reservado

Resolución paso a paso: de la raíz al autoritativo

El resolvedor recursivo pregunta de nivel en nivel hasta obtener la respuesta definitiva.



3 · Raíz responde:

«No lo sé; pregunta a los servidores de .example».

5 · TLD responde:

«Pregunta a los NS de la zona: ns-… de Route 53».

7 · Autoritativo responde:

«A 203.0.113.25, TTL 300».
El resolvedor guarda la respuesta.

Lo que conviene recordar

- El navegador solo hace una pregunta (1); el resolvedor recursivo hace el trabajo largo (2 a 7)
- El resolvedor es el del proveedor de Internet, uno público o el Route 53 Resolver dentro de una VPC
- Con la respuesta en caché, los pasos 2 a 7 se omiten

Fuente: adaptado de AWS Route 53 Developer Guide (How Amazon Route 53 routes traffic) y RFC 1035

Cachés DNS y TTL: la migración sin sorpresas

Lo que guardan los resolvedores no se puede recuperar: se planifica bajando el TTL antes del cambio.

Capas de caché DNS: cada una respeta el TTL del registro

Navegador
caché propia
(un minuto o menos)
TTL hasta 300 s

→

Sistema operativo
resolvedor local
respeta el TTL
TTL hasta 300 s

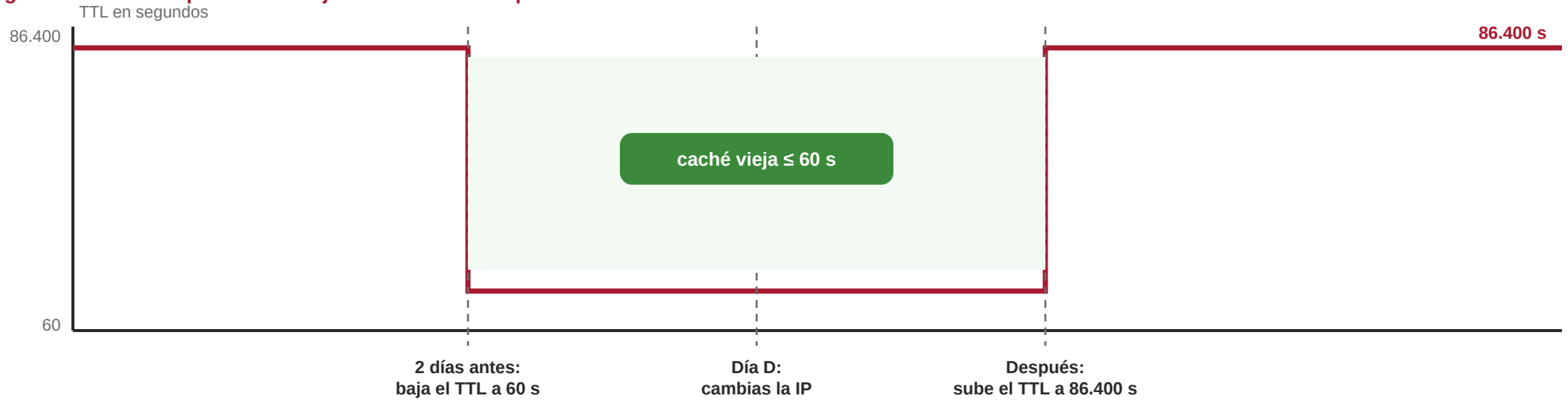
→

Resolvedor recursivo
respeta el TTL;
muchos usuarios lo comparten
TTL hasta 300 s

→

Servidor autoritativo
fuente de verdad;
fija el TTL
TTL lo defines tú

Migración de IP o de proveedor: bajar el TTL con anticipación

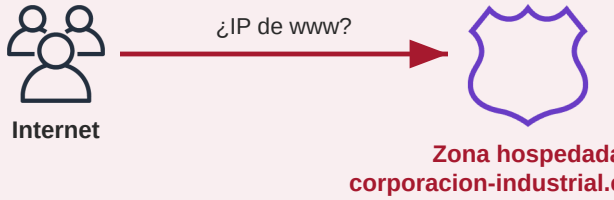


Fuente: adaptado de AWS Route 53 Developer Guide (Values specific to simple records) y buenas prácticas de migración DNS

Zonas hospedadas: pública y privada

La zona pública responde a Internet; la privada responde solo dentro de tus VPC.

Zona hospedada pública



www	A	alias → CloudFront
api	A	alias → balanceador
@	MX	correo corporativo
@	NS	4 servidores de Route 53

Cualquiera en Internet puede consultar estos registros.

Se delega desde el registrador mediante los 4 NS.

Registrar el dominio y alojar la zona son pasos distintos: el registrador debe apuntar a los NS de tu zona.

Zona hospedada privada



bd.interno.corporacion-industrial.example
→ 10.0.3.25 (nombre que no existe en Internet)

Se asocia a una o varias VPC; solo responde dentro de ellas.



Nombres internos estables en lugar de IP privadas

Requiere habilitar nombres y resolución DNS en la VPC.

Fuente: adaptado de AWS Route 53 Developer Guide (Working with hosted zones) · íconos AWS

Los registros DNS más comunes

Cada registro responde una pregunta distinta; la zona es la colección de todos ellos.

Zona corporacion-industrial.example (extracto)

Nombre	Tipo	Valor	TTL
@	A	203.0.113.10	300 s
@	AAAA	2001:db8::10	300 s
www	CNAME	portal.corporacion-industrial.example	300 s
@	MX	10 mail.corporacion-industrial.example	3600 s
@	TXT	"v=spf1 include:_spf.example.net -all"	3600 s
@	NS	ns-xxx.awsdns-xx.org (+ 3 más)	172800 s
@	SOA	ns-xxx.awsdns-xx.org admin…	900 s

¿Para qué sirve cada uno?

A / AAAA
Nombre → dirección IPv4 / IPv6

CNAME
Nombre → otro nombre (no en el ápice)

MX
Dónde se recibe el correo

TXT
Texto: SPF, verificación de dominio

NS
Servidores autoritativos de la zona

SOA
Datos administrativos de la zona

Direcciones de ejemplo reservadas para documentación

Fuente: adaptado de AWS Route 53 Developer Guide (Supported DNS record types) · direcciones RFC 5737 y RFC 3849

Alias frente a CNAME: el ápice del dominio

Route 53 permite que el dominio raíz apunte a CloudFront o a un balanceador mediante un alias.

CNAME estándar

www.corporacion-industrial.example

CNAME → d111111abcdef8.cloudfront.net

1.ª consulta: obtiene el nombre del CNAME

2.ª consulta: resuelve d111…cloudfront.net → IP

corporacion-industrial.example (ápice)

CNAME no permitido en el ápice

El ápice ya tiene SOA y NS; la norma
no admite otros registros junto a un CNAME.

Alias de Route 53

corporacion-industrial.example (ápice)

A (alias) → distribución de CloudFront

Route 53 responde directamente con las IP actuales:

una sola consulta y siempre al día

Destinos posibles



CloudFront



ALB / NLB



Sitio web en S3



API Gateway

Reglas prácticas

· Para apuntar el dominio raíz a un recurso de AWS usa alias; para subdominios hacia un nombre ajeno, CNAME.

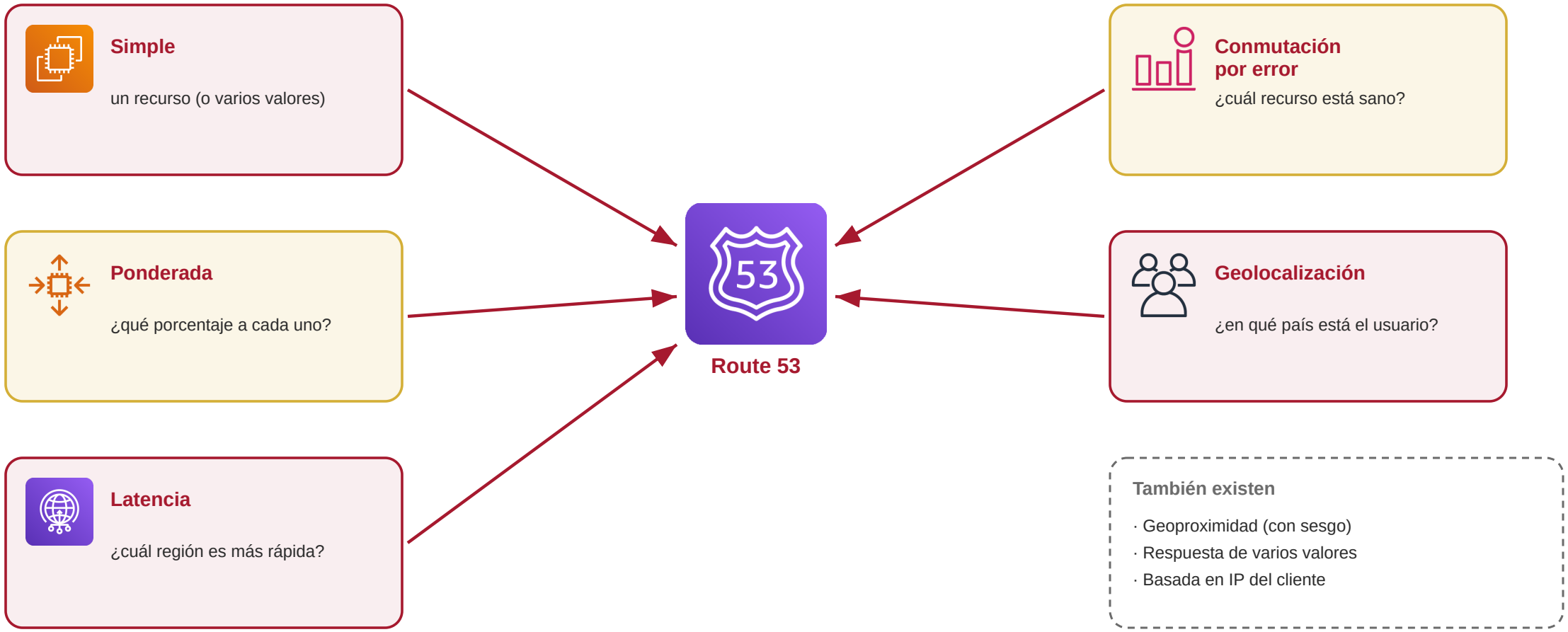
· Las consultas alias a recursos de AWS no se cobran y el alias puede evaluar la salud del destino.

· Alias es una extensión de Route 53; en otros proveedores existen equivalentes con otros nombres (ANAME, CNAME flattening).

Fuente: adaptado de AWS Route 53 Developer Guide (Choosing between alias and non-alias records)

Políticas de enrutamiento de Route 53

Cada política responde una pregunta distinta sobre a dónde enviar a cada usuario.



Se combinan: por ejemplo, latencia entre regiones y failover dentro de cada una.

Fuente: adaptado de AWS Route 53 Developer Guide (Choosing a routing policy) · íconos AWS

Simple y ponderada: del destino único al canario

Con pesos puedes mover un 10 por ciento de usuarios a la versión nueva y subir si todo va bien.

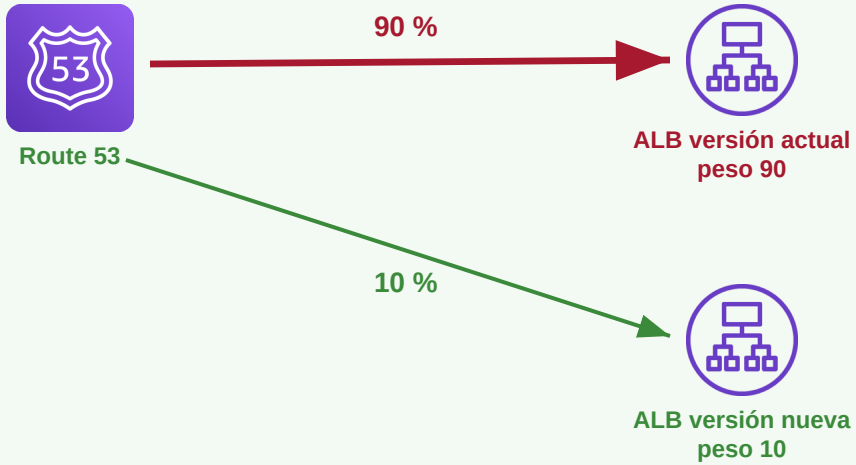
Simple



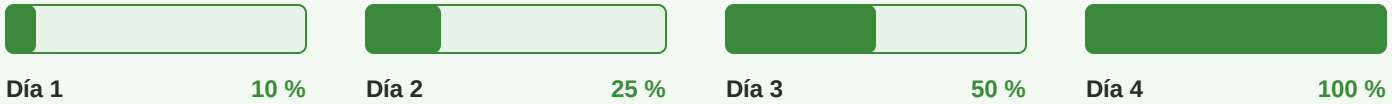
· Un registro, un destino (o una lista de IP devuelta en orden aleatorio)
· Sin chequeos de salud
· Es el punto de partida

Úsala cuando hay un solo recurso que hace el trabajo.

Ponderada · despliegue canario



Avance gradual (la versión nueva recibe…)



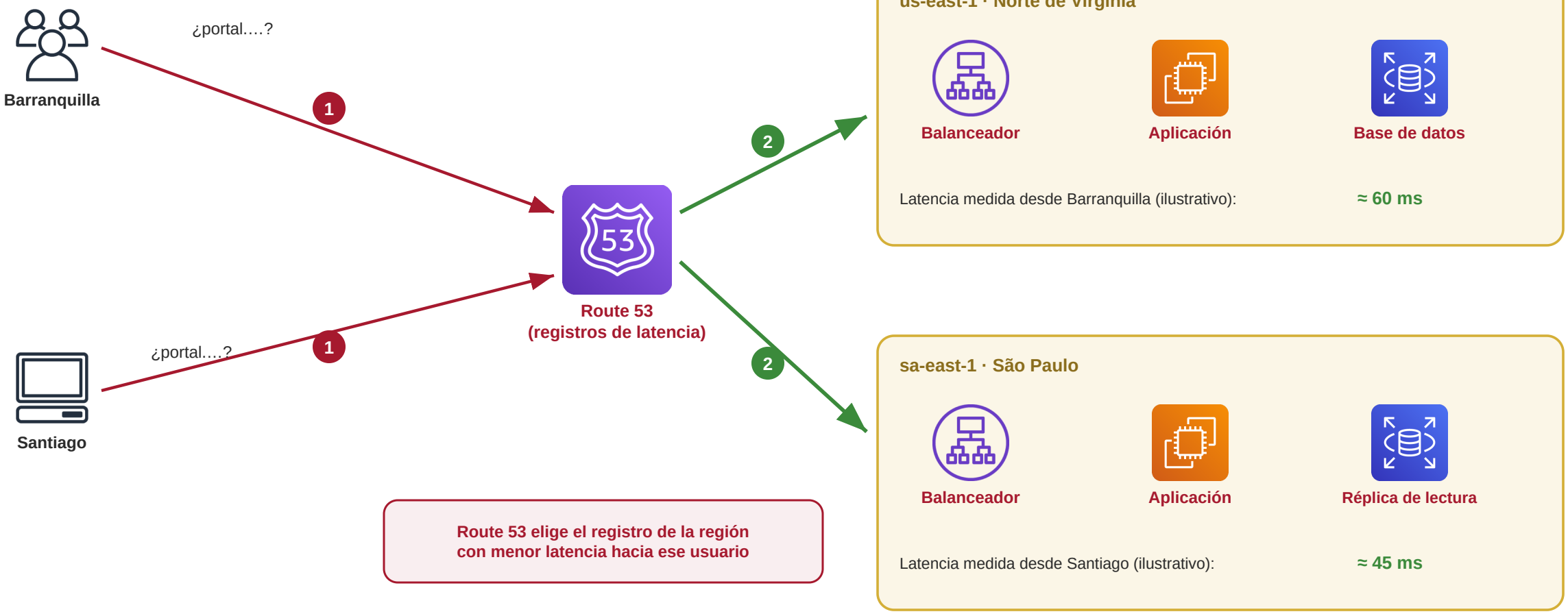
Pesos relativos de 0 a 255; peso 0 retira el recurso. Si algo falla, devuelves el peso a la versión anterior.

Con TTL bajo el reparto se ajusta en minutos.

Fuente: adaptado de AWS Route 53 Developer Guide (Weighted routing) · íconos AWS

Latencia: cada usuario a su región más rápida

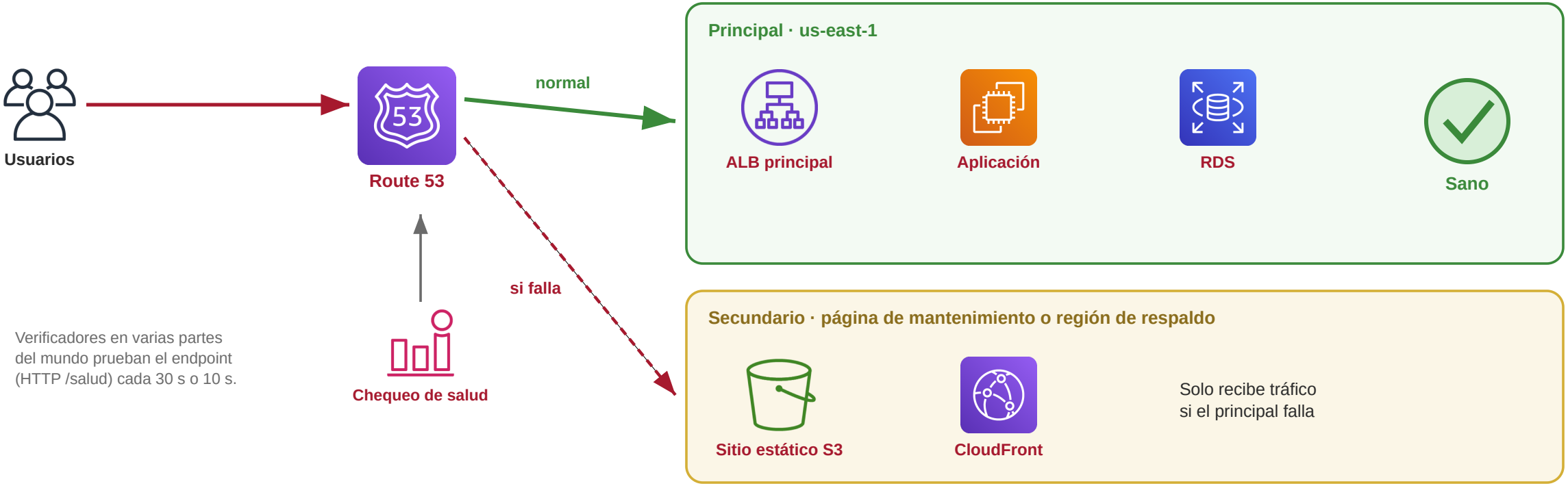
Route 53 compara la latencia desde la red del usuario hacia cada región y responde la mejor.



Fuente: adaptado de AWS Route 53 Developer Guide (Latency-based routing) · cifras ilustrativas

Conmutación por error: salud antes que adivinanzas

Si el chequeo de salud del principal falla, Route 53 deja de responder con él y usa el secundario.



Tiempo hasta la conmutación (peor caso)

detección: 3 fallos × 30 s = 90 s | TTL de 60 s → **≈ 2,5 minutos**

Se acorta con verificación rápida (10 s), umbral menor y TTL bajo (el costo es mayor).

Con alias, puedes marcar «evaluar salud del destino» y reutilizar la salud del recurso AWS.

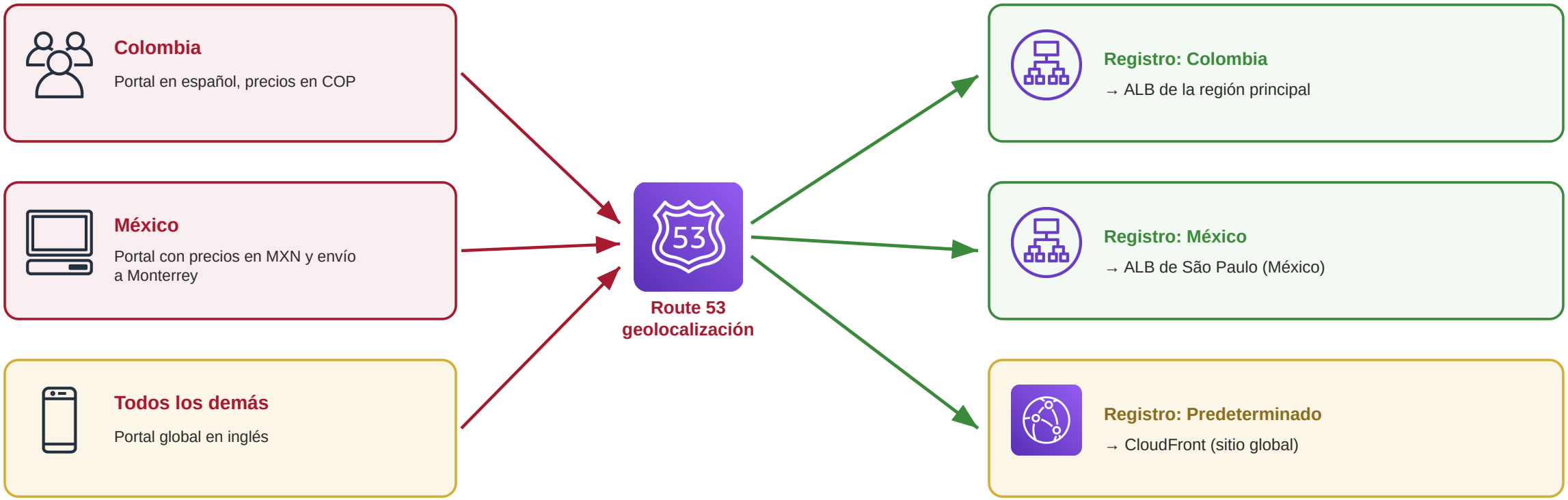
Fuente: adaptado de AWS Route 53 Developer Guide (Configuring DNS failover) · tiempos ilustrativos

ENRUTAMIENTO

Geolocalización: decidir por el país del usuario

Se usa cuando importa quién es el usuario (idioma, moneda, normativa), no qué tan rápido responde.

Según el país del usuario



Sin registro predeterminado, un usuario cuyo país no coincide no recibe respuesta.

Geolocalización ≠ latencia: aquí importa la regla de negocio (idioma, moneda, licencias, normativa), no el tiempo.

Fuente: adaptado de AWS Route 53 Developer Guide (Geolocation routing) · ejemplo docente

CASO APLICADO

Corporación Industrial: el diseño DNS completo

Alias para el portal, latencia con salud para la API y failover para el soporte.



Zona privada interna: bd.interno… → RDS · cache.interno… → ElastiCache

Buenas prácticas aplicadas: alias en lugar de CNAME, TTL de 60 s en registros con salud y 300 s en el resto, chequeos de salud con alarma en CloudWatch y TTL bajado dos días antes de cualquier migración.

Fuente: caso docente Corporación Industrial del Caribe · adaptado de AWS Route 53 Developer Guide

CIERRE

Lo que queda de la Parte 3

El DNS decide a dónde va cada usuario; con políticas y salud se vuelve una herramienta de arquitectura.



Resolución y TTL

Raíz, TLD y autoritativo; cachés en cada capa.

1



Zonas y registros

Pública para Internet, privada para la VPC; alias en el ápice.

2



Políticas de enrutamiento

Ponderada, latencia, failover y geolocalización.

3



Todo se combina

DNS decide el destino; CDN y origen entregan.

4

Pregunta para pensar antes de la Parte 4

El borde y el DNS ya aceleran lo que ve el cliente. ¿Y lo que pasa dentro del servidor?

Cada página dinámica sigue consultando la base de datos: ahí aparece la caché de aplicación.

Ideas clave

- El DNS es la primera caché
- Baja el TTL antes de migrar
- Alias para el dominio raíz
- Política según la pregunta

Siguiente: Parte 4

Caché de aplicación y base de datos con ElastiCache

SEMANA 12 · CACHÉ Y DISTRIBUCIÓN DE CONTENIDO

Caché de aplicación y de base de datos con ElastiCache

Parte 4 de 4 · Unidad 3

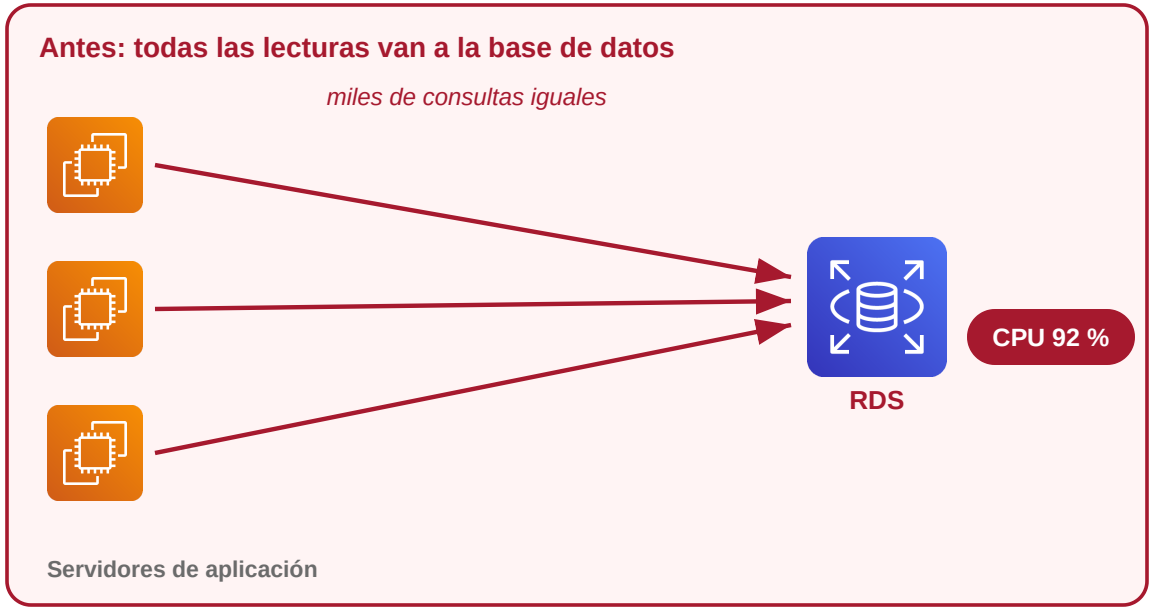
Ing. Rodolfo Cañas Cervantes
Universidad de la Costa (CUC) · Ingeniería de Sistemas
Semana 12 · Periodo 2026-2 · Barranquilla, Colombia

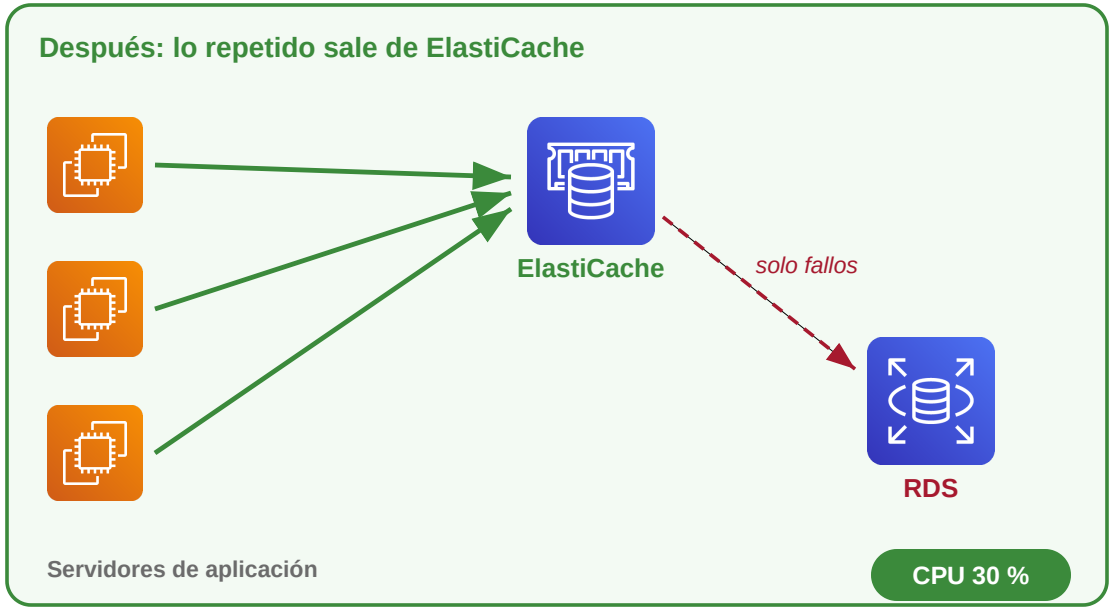
Redis o Memcached, carga diferida o escritura directa, y la arquitectura final del portal.

ENCUADRE

La base de datos es el cuello de botella

Miles de consultas idénticas castigan a RDS; una caché en memoria las responde en microsegundos.





2.400 → 240
consultas por segundo a la base de datos

480 → 45 ms
latencia p95 de la página de catálogo

2 → 1
tamaño de instancia RDS necesario

Cifras ilustrativas de una prueba de carga del portal; en un proyecto real se miden antes y después.

Escalar la base de datos hacia arriba es caro; evitarle trabajo repetido es barato.

ElastiCache: caché en memoria administrada

Elige un motor y un modo de despliegue; AWS se encarga del hardware, los parches y la conmutación.

Valkey y Redis OSS

Estructuras de datos, réplicas, failover, persistencia

Memcached

Clave-valor simple, multihilo, sin réplicas

Amazon ElastiCache

Caché administrada en memoria sin parches ni hardware que operar

Dos formas de desplegar

· Serverless: sin elegir nodos; escala sola según uso de memoria y solicitudes
· Clústeres de nodos: eliges tipo, número de nodos, réplicas y particiones

Servicios cercanos

DAX: caché integrada para DynamoDB
(sin cambiar el código del cliente)

MemoryDB: compatible con Redis y durable

Cuidado con el nombre de los motores

· ElastiCache admite Valkey (código abierto, compatible con Redis) y Redis OSS; para proyectos nuevos se recomienda Valkey.
· Las versiones recientes de Redis cambiaron de licencia: por eso Valkey nació como bifurcación de la Linux Foundation.
· En la práctica, el código cliente y los comandos que veremos funcionan igual con Redis OSS y Valkey.

Fuente: adaptado de Amazon ElastiCache User Guide y anuncios de AWS sobre Valkey · íconos AWS

MOTORES

Redis (Valkey) frente a Memcached

Redis ofrece más funciones; Memcached es más simple y aprovecha varios núcleos por nodo.

	Redis / Valkey	Memcached
Estructuras de datos	✓ listas, conjuntos, hashes, conjuntos ordenados	✗ solo cadenas de texto o binarios
Réplicas y failover Multi-AZ	✓ sí, automático	✗ no: un nodo que falla pierde su parte
Persistencia y respaldo	✓ instantáneas y respaldos	✗ ninguna: es volátil
Escalado de lectura y escritura	✓ particiones (cluster mode) y réplicas de lectura	✓ nodos y particionado en el cliente
Multihilo	✗ principalmente un hilo por comando	✓ sí, aprovecha muchos núcleos
Pub/sub, transacciones, scripts	✓ sí	✗ no

Valores generales para ElastiCache; revisa la guía del servicio para la versión exacta del motor que uses.

Fuente: adaptado de Amazon ElastiCache User Guide (Comparing Memcached and Redis OSS) · íconos AWS

¿Redis o Memcached? Un árbol de decisión

Si necesitas durabilidad, réplicas o estructuras, Redis; si solo cachea objetos, Memcached.

¿Necesitas replicación, persistencia, estructuras avanzadas o failover?

Sí



Elige Redis / Valkey

Ejemplos: sesiones que no deben perderse, tablas de clasificación, contadores, colas ligeras, datos con réplica Multi-AZ

Si dudas, empieza con Redis / Valkey

Cubre más casos, y la diferencia de costo es pequeña frente a rehacer la arquitectura.

No

Entonces pregunta:

¿Es solo caché de objetos que puedes reconstruir desde la base de datos y quieres multihilo y escalado horizontal sencillo?

Sí



Memcached es suficiente

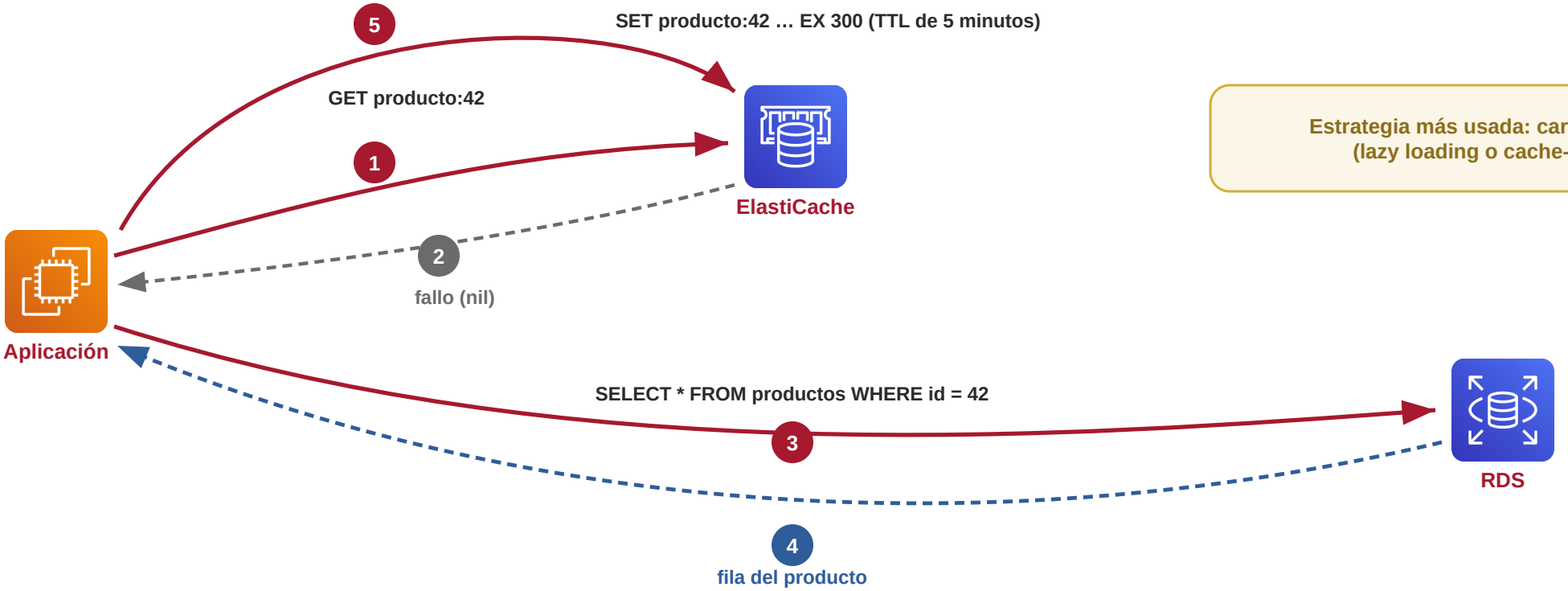
Ejemplo: fragmentos HTML y resultados de consulta

Para el portal de la Corporación: Redis / Valkey, porque guarda sesiones y requiere Multi-AZ.

Fuente: adaptado de Amazon ElastiCache User Guide y AWS Database Caching Strategies · criterio docente

Carga diferida: la aplicación llena la caché

La aplicación consulta primero la caché; si falla, lee la base de datos y guarda el resultado.



Estrategia más usada: carga diferida (lazy loading o cache-aside)

Ventajas

- Solo se cachea lo que alguien pidió
- Si la caché cae, la aplicación sigue (con la BD)

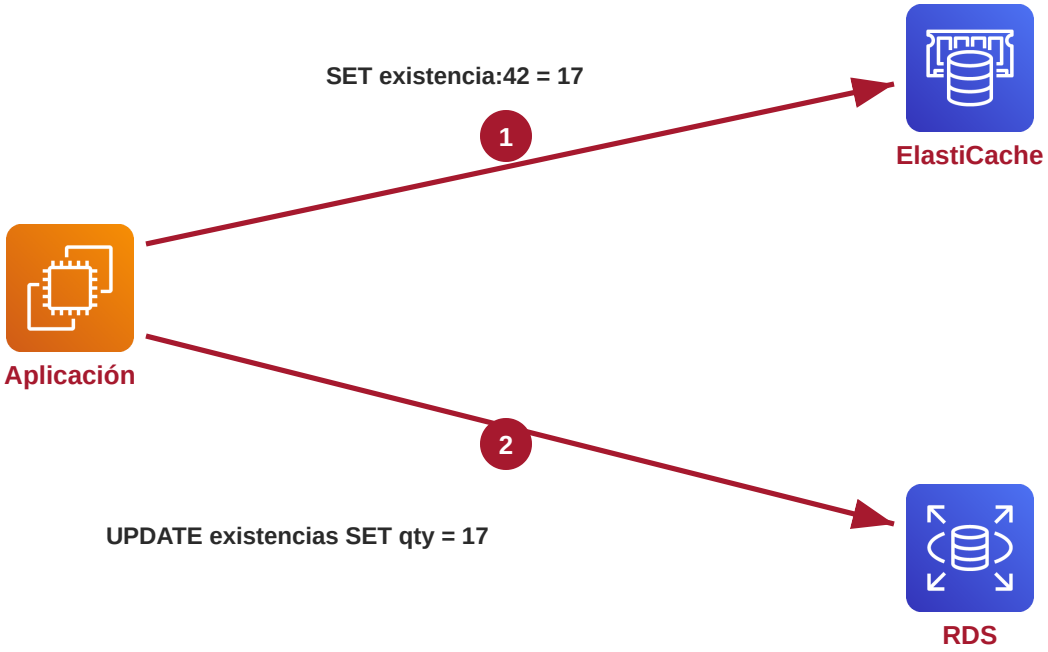
Costos

- La primera lectura es lenta (fallo)
- Los datos pueden estar viejos hasta que venza el TTL

Fuente: adaptado de AWS Database Caching Strategies Using Redis (Lazy loading) · íconos AWS

Escritura directa: la caché siempre al día

Cada vez que se escribe en la base de datos se actualiza también la caché.



Cada escritura actualiza la base de datos y la caché

Las lecturas encuentran siempre el dato vigente y rápido

Ventajas

- Datos frescos
- Lecturas siempre aciertan
- Sin penalización en la lectura

Costos

- Escribir es más lento
- Guarda datos que quizá nadie lea
- Si falla un paso hay que repararlo

Úsala para datos que se leen mucho y cambian poco, como existencias consultadas por todos los vendedores.

Añade un TTL para que lo que nadie lee no ocupe memoria indefinidamente, y define el orden de escritura (primero BD).

Variante: escritura diferida (write-behind): la caché escribe en la BD después; es más rápida y más riesgosa.

Fuente: adaptado de AWS Database Caching Strategies Using Redis (Write-through) · íconos AWS

ESTRATEGIAS

Carga diferida frente a escritura directa

La carga diferida arranca fría y puede quedar vieja; la escritura directa está al día pero guarda de más.

Carga diferida (lazy loading)

t0 · arranque

A	B	C	D	E	F

t1 · se leen A, C y F

A	B	C	D	E	F
vigente		vigente			vigente

t2 · cambia A en la BD

A	B	C	D	E	F
obsoleto		vigente			vigente

Frío al inicio · solo lo leído · A queda viejo hasta el TTL

vigente · obsoleto · sin dato

Escritura directa (write-through)

t0 · arranque

A	B	C	D	E	F

se escriben A–F

A	B	C	D	E	F
vigente	vigente	vigente	vigente	vigente	vigente

t2 · cambia A en la BD

A	B	C	D	E	F
vigente	vigente	vigente	vigente	vigente	vigente

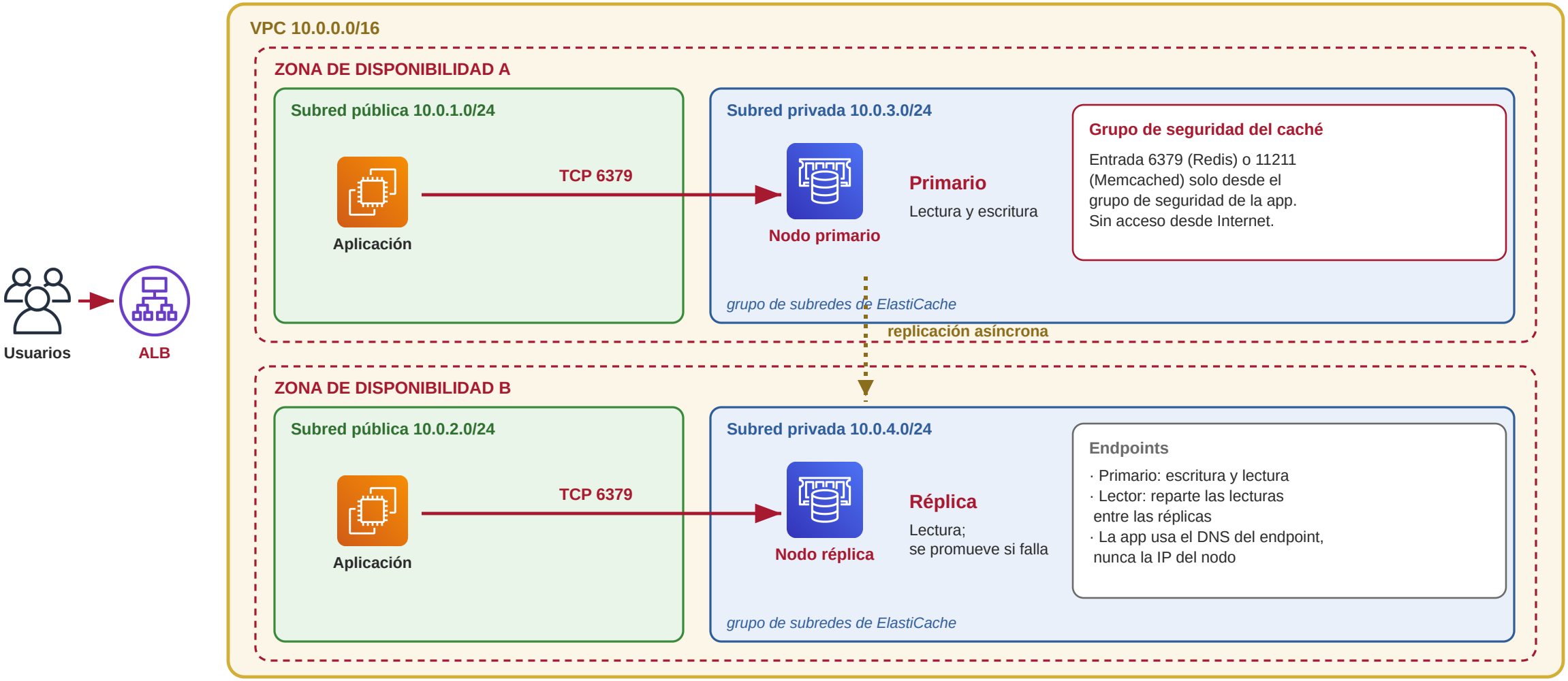
Todo al día · incluye lo que nadie lee (B, D, E)

En la práctica se combinan: carga diferida para lecturas + escritura directa en lo crítico + TTL en todo.

Fuente: adaptado de AWS Database Caching Strategies Using Redis · ejemplo ilustrativo

ElastiCache dentro de la VPC: subredes y réplicas

El caché vive en subredes privadas de dos zonas y solo acepta tráfico del grupo de seguridad de la aplicación.



Fuente: adaptado de Amazon ElastiCache User Guide (Subnets and subnet groups) y plantilla VPC del curso · íconos AWS

Escalar la caché: particiones y nodos

Redis reparte las claves en ranuras hash entre particiones; Memcached lo hace desde la biblioteca cliente.

Redis con particiones (cluster mode)

Partición 1

ranuras hash 0 – 5460



Primario — Réplica

zona A · zona B

las claves se reparten por CRC16(clave) mod 16384

Partición 2

ranuras hash 5461 – 10922



Primario — Réplica

zona A · zona B

las claves se reparten por CRC16(clave) mod 16384

Partición 3

ranuras hash 10923 – 16383



Primario — Réplica

zona A · zona B

las claves se reparten por CRC16(clave) mod 16384

· 16.384 ranuras hash divididas entre las particiones

· Se agregan particiones para escalar escritura y memoria; réplicas para lectura

Memcached en varios nodos



con biblioteca cliente

Aplicación

el cliente reparte las claves entre los nodos



Nodo 1 — Nodo 2 — Nodo 3

· Descubrimiento automático de nodos

· Si un nodo cae, se pierden sus claves

Escalar sin interrupciones

· Escala vertical: cambias el tipo de nodo (más memoria y CPU). · Escala horizontal: más particiones (Redis) o más nodos (Memcached).

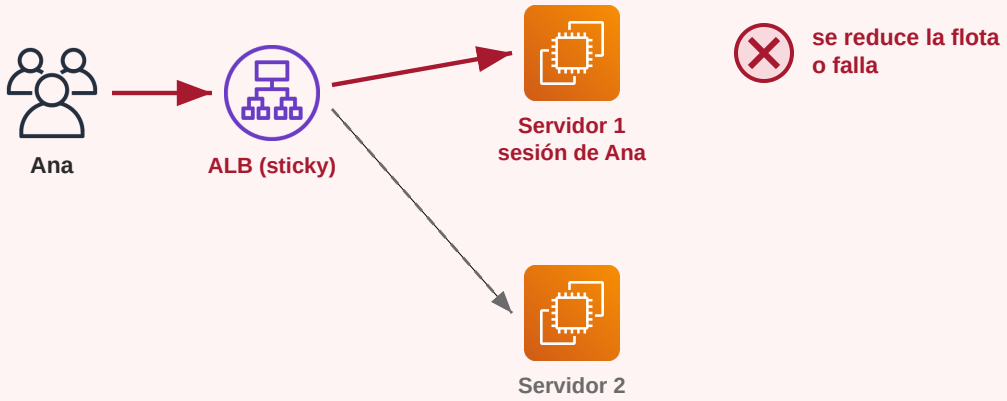
· ElastiCache Serverless hace estas decisiones por ti, con un costo por uso de memoria y solicitudes.

Fuente: adaptado de Amazon ElastiCache User Guide (Scaling) y Redis Cluster Specification · íconos AWS

Sesiones en Redis: servidores sin estado

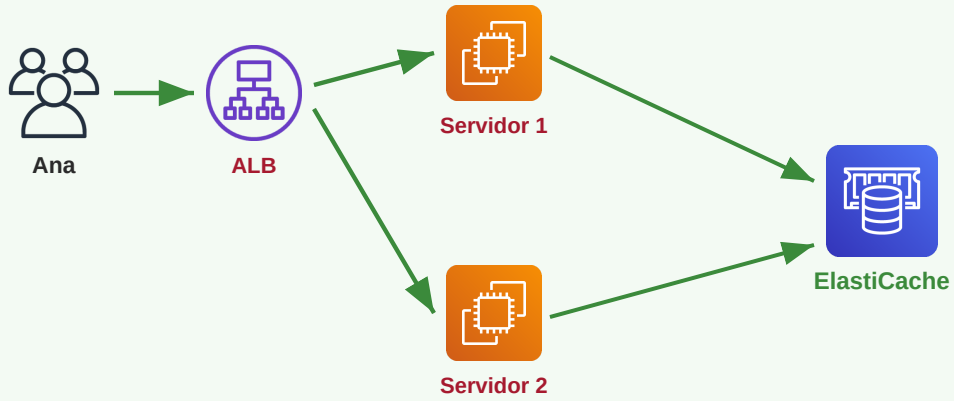
Sacar la sesión del servidor permite escalar y reemplazar instancias sin cerrar la sesión a nadie.

Sesión en la memoria del servidor



· Ana pierde el carrito y vuelve a iniciar sesión
· El balanceador debe pegarse (sticky) al servidor
· Escalar hacia adentro cierra sesiones activas

Sesión en ElastiCache



sesion:9f3a… → {usuario: 4501, carrito: [3 ítems]}
TTL de 30 minutos, renovado en cada solicitud

· Cualquier servidor atiende a Ana
· La flota escala sin cerrar sesiones

Servidores sin estado + sesión externa = elasticidad real. Es el patrón base del Módulo 10 (alta disponibilidad).

Fuente: adaptado de AWS Well-Architected (Fiabilidad) y Amazon ElastiCache for Redis session store · íconos AWS

Memoria, evicción y métricas

Una caché llena no falla: borra claves. Las métricas muestran si lo hace demasiado.

Memoria del nodo (ejemplo: 6,38 GiB)



Al llenarse, el motor elimina claves (evicción) según su política

allkeys-lru	la menos usada de todas
volatile-lru	la menos usada entre las que tienen TTL (valor predeterminado en ElastiCache for Redis)
noeviction	no borra: rechaza escrituras al llenarse

Memcached elimina siempre las claves menos usadas (LRU) al llenarse.

Métricas de CloudWatch para vigilar

CacheHits y CacheMisses
→ tasa de aciertos

Evictions
claves eliminadas por falta de memoria

DatabaseMemoryUsagePercentage
memoria usada del nodo

EngineCPUUtilization
CPU del hilo del motor

CurrConnections
conexiones de las apps

Señales de alerta y qué hacer

· Evictions creciendo: faltan memoria o TTL; escala el nodo o agrega particiones, o reduce el tamaño de los valores.

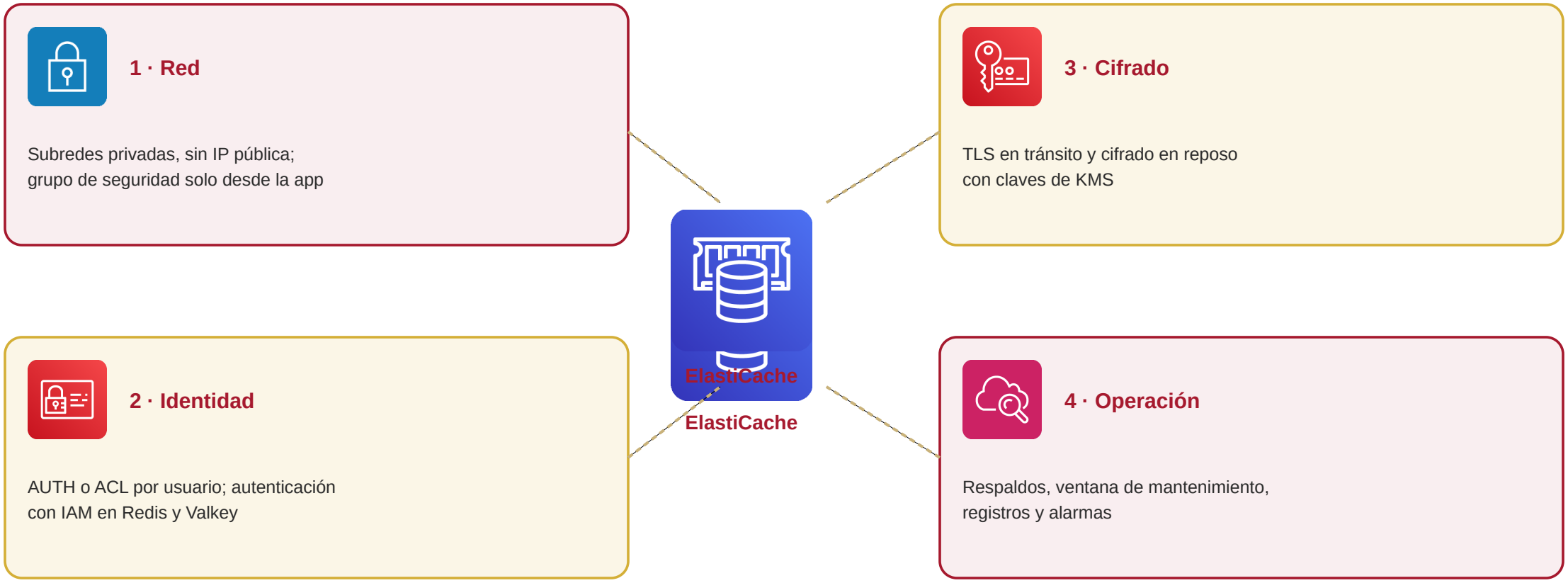
· Aciertos bajando: revisa TTL, claves y cambios de código. · CPU alta: demasiados comandos pesados; evita KEYS * y usa SCAN.

· Crea alarmas de CloudWatch sobre Evictions, memoria y CPU; la caché también necesita monitorización.

Fuente: adaptado de Amazon ElastiCache User Guide (Metrics, maxmemory-policy) · valores de ejemplo

Asegurar ElastiCache: red, identidad y cifrado

Una caché guarda sesiones y datos personales: se protege como una base de datos.

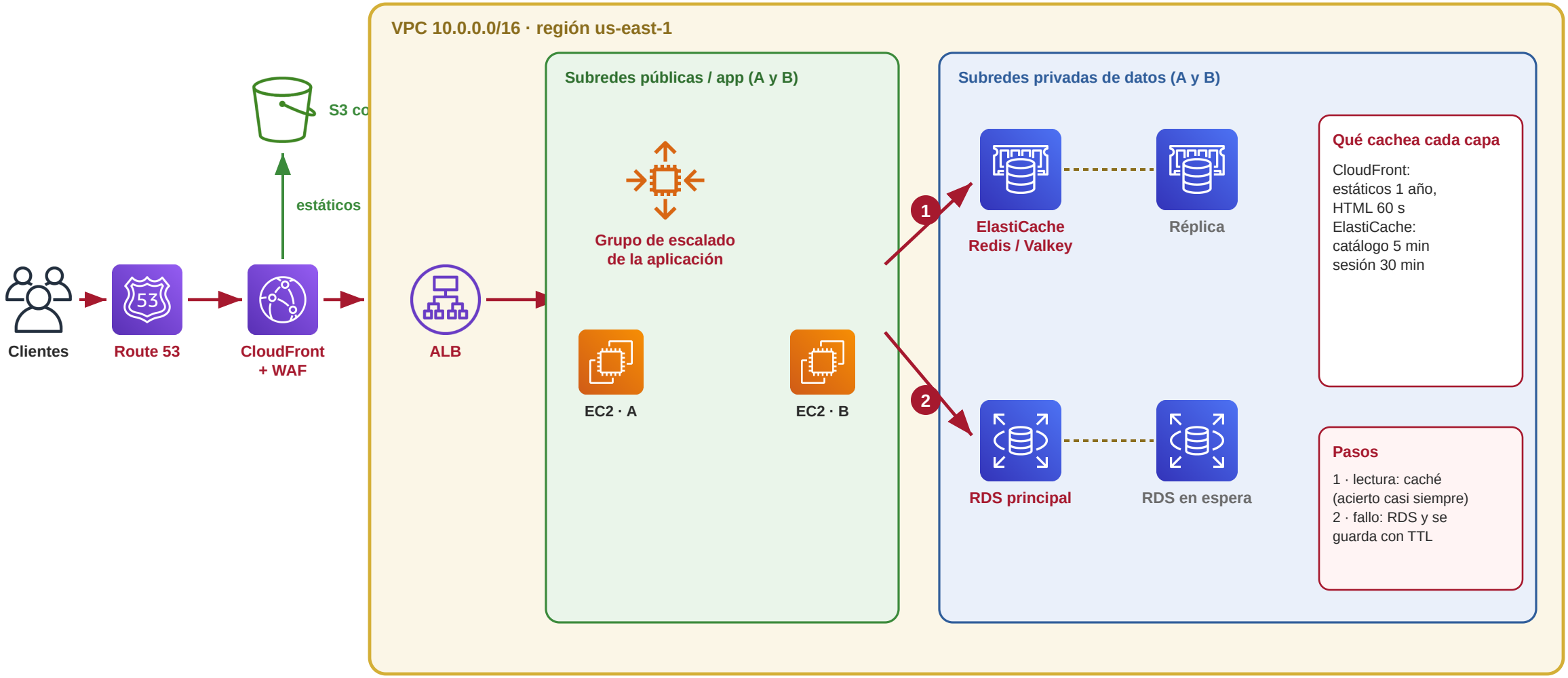


Error frecuente: un caché abierto a 0.0.0.0/0 o sin autenticación. En una caché no hay datos «de poco valor»: guardan sesiones, tokens y datos personales sujetos a la Ley 1581 de 2012.

Fuente: adaptado de Amazon ElastiCache User Guide (Security) · AWS Well-Architected, pilar Seguridad · Ley 1581 de 2012

Arquitectura final del portal de la Corporación

Cuatro niveles de caché trabajan juntos: navegador, CDN, ElastiCache y la base de datos Multi-AZ.



Fuente: caso docente Corporación Industrial del Caribe · adaptado de AWS Architecture Center · íconos AWS

CIERRE

Semana 12 en cuatro ideas

Cachear en cada nivel convierte a la arquitectura en rápida, barata y resistente.

Semana 12 en cuatro ideas



Parte 1
Por qué cachear

Latencia, hit ratio, TTL y qué no cachear.



Parte 2
CDN

Borde, clave de caché, versionado, HTTPS, OAC y WAF.



Parte 3
DNS

Resolución, alias y políticas: latencia, failover, geolocalización.



Parte 4
ElastiCache

Redis o Memcached, carga diferida, sesiones y seguridad.

Para estudiar

- Revisa las métricas CacheHitRate (CloudFront) y CacheHits/CacheMisses (ElastiCache)
- Practica: calcula la latencia media con h = 95 %
- Guía del estudiante AWS Academy, Módulo 12

Idea de la semana

Acerca la copia a quien la usa, mide los aciertos y respeta frescura y privacidad.

Siguiente: Semana 13

Arquitecturas desacopladas: colas, pub/sub, eventos y patrones de resiliencia.

Recordatorio académico

Último día de retiro de asignaturas: 1 de noviembre.

Esta semana no tiene laboratorio de desafío evaluado.

Pregunta de cierre

¿Qué dato de tu proyecto de aula tolera 60 segundos de obsolescencia a cambio de responder 10 veces más rápido?

Fuente: síntesis de la Semana 12 · AWS Academy Cloud Architecting, Módulo 12